2026-2 중간고사 대비

프로그래밍 언어론 · 중간 학습 노트

단원 7개 · 레슨 29개 · 트레이너 학습 탭과 같은 내용

U1 Ch.1 기본사항 (Preliminaries)

L1-1 · 왜 언어론을 배우나 · 프로그래밍 도메인

L1-2 · 평가 기준 ① 판독성

L1-3 · 평가 기준 ② 작성력 · 신뢰성 · 비용

L1-4 · 설계에 대한 영향 · 언어 범주 · 설계 절충

L1-5 · 구현 방법 · 전처리기 · 프로그래밍 환경

U2 Ch.3 구문 기술 (BNF · 파스 트리 · 모호성)

L2-1 · 구문 vs 의미론 · 기본 용어

L2-2 · BNF와 유도

L2-3 · 파스 트리 · 모호성

L2-4 · 우선순위 · 결합성 · if-else 모호성

L2-5 · EBNF

U3 Ch.3 속성 문법 · 의미론 (최약 전조건)

L3-1 · 정적 의미론 · 속성 문법

L3-2 · 동적 의미론 3가지 · 연산 · 표기 의미론

L3-3 · 공리 의미론 · 배정문의 최약 전조건

L3-4 · 결과 규칙 · 시퀀스 · 선택문

L3-5 · 루프 불변자 · 정확성 증명

U4 Ch.4 어휘 분석

L4-1 · 구문 분석의 두 부분 · 분리하는 이유

L4-2 · 어휘 분석기 · 상태 다이어그램

L4-3 · front.c 따라가기

U5 Ch.4 파싱 문제 · 하향식 파싱

L5-1 · 파싱 문제 · 하향식 vs 상향식

L5-2 · 재귀 하강 파서

L5-3 · LL 문법: 좌순환 · 집합쌍 공통 · 좌추출

U6 Ch.4 상향식 파싱 (핸들 · LR)

L6-1 · 핸들 · 어구 · 단순 어구

L6-2 · Shift-Reduce · LR 파서 구조

L6-3 · LR 파싱 추적 연습

U7 Ch.5 이름 · 바인딩 · 영역

L7-1 · 이름 · 변수의 6가지 속성

L7-2 · 바인딩 · 바인딩 시간 · 타입 바인딩

L7-3 · 기억장소 바인딩과 생존 기간

L7-4 · 정적 영역 · 블록 · 전역 영역

L7-5 · 동적 영역 · 영역 vs 생존 기간 · 참조 환경

U1 · L02 Preliminaries.pdf

Ch.1 기본사항 (Preliminaries)

언어를 "평가"하는 기준과 언어를 "구현"하는 방법. 서술형 3점짜리가 여기서 줄줄이 나와. 영어 용어 ↔ 한글 용어(판독성·작성력·신뢰성·비용)를 짝으로 외워.

왜 언어론을 배우나 · 프로그래밍 도메인

교안 범위: L02 p.3–7

§ 언어 개념을 공부하는 이유

교안에 6+2개가 나와. 시험에 통째로 나올 확률은 낮지만, 키워드는 알아둬.

- **표현 능력 증가** (Increased ability to express ideas): 아는 언어가 생각의 한계를 정한다. 다른 언어의 기능을 알면 지금 언어에서도 흉내 낼 수 있다.
- **적절한 언어 선택 능력** (Improved background for choosing appropriate languages)
- **새 언어 학습 능력 증가** (Increased ability to learn new languages)
- **구현의 중요성 이해** (Better understanding of significance of implementation): 구현을 알면 언어가 *왜* 그렇게 설계됐는지 보이고, 더 효율적으로 쓴다.
- **이미 아는 언어를 더 잘 쓰기** (Better use of languages already known)
- **컴퓨팅 전반의 발전** (Overall advancement of computing)
- (추가) 언어 설계·컴파일러 구조에 대한 깊은 이해, 코드 유지보수성 향상

한 줄 요약(교안 Summary): 다양한 구문을 쓰는 능력↑, 언어를 똑똑하게 고름, 새 언어 배우기 쉬움.

수업 메모 02주-1
생성형 AI 시대라 코드를 정확히 '읽고 분석하는' 능력이 더 중요해졌다고 강조. 구현 이해의 예로 C 2차원 배열을 들었음: C는 행 우선(row-major)이라 행 순서로 돌면 연속된 메모리를 캐시로 읽어 빠르고, 열 순서로 돌면 메모리를 더 자주 읽어 느리다.

§ 프로그래밍 도메인

도메인 ↔ 대표 언어 ↔ 핵심 특징을 한 세트로 외우면 돼.

도메인	대표 언어	특징
과학 응용 (Scientific)	**Fortran** (FORmula TRANslation)	대량의 부동소수점 계산, 배열
사무 응용 (Business)	**COBOL**	보고서 생성, 십진수·문자 처리
인공지능 (AI)	**LISP** (LISt Processing)	숫자보다 기호(symbol) 처리, 연결 리스트, 코드=데이터 (homoiconicity)
시스템 프로그래밍	**C**	계속 실행되므로 효율이 중요
웹 소프트웨어	HTML, PHP, Java 등	마크업 + 스크립팅 + 범용 언어의 잡다한 조합

함정: LISP의 핵심은 "기호 처리 + 리스트". 수치 계산 = Fortran, 십진수 = COBOL.

수업 메모 02주-1
COBOL: 부동소수점은 근사치인데, COBOL은 십진수를 자리별로 그대로 인코딩해서 정확하게 다룬다 (사무용이라 돈 계산이 정확해야 함). LISP의 동형성: (A B C D) 같은 리스트가 데이터일 수도 있고 코드(함수 호출)일 수도 있어서, 코드로 코드를 만들 수 있다. 시스템 프로그래밍: C는 배열 범위를 실행 시간에 검사하지 않아 오버헤드가 없다 (Java는 검사함).

평가 기준 ① 판독성

교안 범위: L02 p.8–13

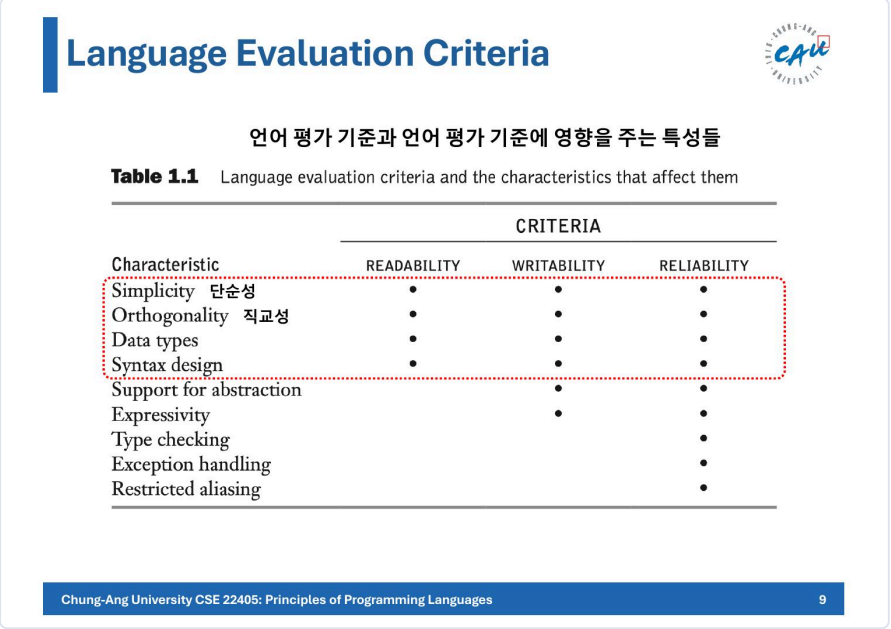
§ 언어 평가 기준 4가지

기준	한글	한 줄 정의
Readability	**판독성**	프로그램을 얼마나 쉽게 읽고 이해할 수 있나
Writability	**작성력**	주어진 도메인의 프로그램을 얼마나 쉽게 작성할 수 있나
Reliability	**신뢰성**	모든 조건에서 명세대로 동작하나 (conformance to specifications)
Cost	**비용**	교육·작성·컴파일·실행·유지보수 등의 총비용

> 시험 포인트: 24 중간 1번이 정확히 "평가 기준 4가지와 각 기준을 간략히". 정의 한 줄씩만 쓰면 3점.

교안 표(p.9)의 관계: **단순성, 직교성, 데이터 타입, 구문 설계**는 판독성·작성력·신뢰성에 모두 영향. **추상화 지원, 표현력**은 작성력·신뢰성. **타입 검사, 예외 처리, 제한된 별칭**은 신뢰성만.

> **수업 메모** 02주-1
> 평가 기준은 언어를 쓰는 목적(도메인)에 따라 달라진다고 강조. 예: GUI 코딩엔 Visual Basic이 편하고, 시스템 프로그래밍에 Python은 실행 오버헤드 때문에 고르지 않는다. 4가지 중 앞의 3개(판독성·작성력·신뢰성)가 언어 특성과 직접 연결되는 기준이다.



L02 p.9 · 언어 평가 기준과 영향을 주는 특성 (표 1.1)

§ 판독성이 왜 중요해졌나

- 유지보수가 SW 생명주기의 큰 부분으로 인식됨 → 유지보수 용이성은 판독성이 좌우.
- 관점이 **기계 중심 → 사람 중심**으로 바뀜.
- 판독성은 그 언어의 문제 도메인 맥락에서 판단해야 한다.

§ 전반적 단순성 (Overall simplicity)

기능·구조가 적은 언어가 읽기 쉽다. 단순성을 해치는 두 가지:

- **특징 다중성** (feature multiplicity): 같은 일을 하는 방법이 여러 개.
  예) `cnt = cnt + 1;` `cnt += 1;` `cnt++;` `++cnt;`
- **연산자 중복** (operator overloading): 하나의 연산자 기호가 여러 의미.

> **수업 메모** 02주-1
> 판독성에 가장 큰 영향을 주는 게 전반적 단순성. 언어가 크면 사람마다 아는 부분집합이 달라서 남이 쓴 코드를 이해하기 어렵다. 연산자 중복 예: C의 &는 비트 AND이면서 주소 연산자, +는 정수+정수와 실수+실수에서 실제 연산이 다름, C++에서 벡터에 + 를 오버로딩해 이상하게 쓰면 읽기 어려워짐.

---

§ 직교성 (Orthogonality)

적은 수의 기본 구조를 조합한 모든 경우가 합법적이고 의미가 있는 성질.
직교성↑ → 규칙의 예외↓ → 규칙성↑ → 배우기·읽기 쉬움.

C의 비직교적 예:

- 구조체(record)는 함수에서 반환할 수 있지만 배열은 못 한다.
- 배열 원소는 void나 함수를 제외한 모든 타입이 될 수 있다.

직교적 예: VAX의 `ADDL op1, op2` (피연산자가 레지스터든 메모리든 다 됨).

너무 많은 직교성도 문제: 모든 것이 값을 갖는 식이 되면

```
x := (if a > b then a := b else b := a fi);
```

같은 이상한 조합도 합법이 되어 오히려 읽기 어렵고, 오류가 안 잡힌다.

> 시험 포인트: "직교성이란?" → "적은 기본 구조들의 모든 조합이 합법적이고 의미 있음" + C 배열 반환 불가 예.

> **수업 메모** 02주-1
> 좌표축(x, y가 서로 독립이라 아무 조합이나 가능)에 비유. VAX의 ADDL은 레지스터·메모리 모든 조합을 한 명령으로 지원(직교적)인데, IBM 계열은 레지스터+메모리, 레지스터+레지스터를 다른 명령(A, AR)으로 나눠야 해서 예외가 생긴다(비직교적). C에서 구조체는 반환되지만 배열은 안 되는 것도 '예외'의 예.

---

§ 데이터 타입과 구문 설계

데이터 타입: 타입·자료구조를 정의할 충분한 장치가 있으면 읽기 쉽다.
예) Boolean이 없으면 `timeOut = 1` (뭔 뜻?) vs `timeOut = false` (명확). C89엔 bool이 없었고 C99의 `stdbool.h`에서 생김.

구문 설계가 판독성에 주는 영향 3가지:

- **특수어** (special words): `while`, `for` 같은 단어. 복합문 형성 방법도 포함 (C는 `{}`, Ada는 `end if`, `end loop` → Ada가 더 읽기 쉬움).
- **형식과 의미** (form and meaning): 의미가 형식(구문)에서 바로 드러나야 한다.
  반례: C의 `static`은 위치에 따라 의미가 다르다. 함수 안 → 컴파일 시간에 생성(호출마다 새로 안 생김). 함수 밖 → 그 파일에서만 보임.

수업 메모 02주-1

C의 static: 함수 안에 선언하면 다시 호출돼도 값이 유지되고, 함수 밖에 선언하면 그 파일 안에서만 보인다. 같은 예약어가 위치에 따라 뜻이 달라서 '형식과 의미' 원칙 위반.

평가 기준 ② 작성력 · 신뢰성 · 비용

교안 범위: L02 p.14–18

§ 작성력 (Writability)

판독성에 영향 주는 특성 대부분이 작성력에도 영향. 도메인 맥락 중요.

- **단순성과 직교성**: 적은 기본 구조 + 적은 조합 규칙. 장점: 단순한 집합만 배우면 됨. 단점: 직교성이 너무 크면 오류가 탐지 안 됨.
- **추상화 지원**: 세부를 무시하고 복잡한 구조를 정의·사용.
- **표현력** (Expressivity): 강력한 연산자로 작은 프로그램이 많은 계산을 함. 예) APL의 `A+.×B`, C의 `count++`가 `count = count + 1`보다 편함.

§ 신뢰성 (Reliability)

모든 조건에서 명세대로 수행하면 신뢰성이 있다. 영향 주는 특성:

- **타입 검사** (type checking): 오류는 일찍 발견할수록 고치는 비용이 싸다. 컴파일 시간 검사 > 실행 시간 검사. Java는 거의 모든 변수·식의 타입을 컴파일 시간에 검사.
- **예외 처리** (exception handling): 실행 시간 오류를 가로채 시정 조치.
- **별칭** (aliasing): 같은 메모리 위치를 두 개 이상의 이름(참조 방법)으로 접근 → 신뢰성 저하.
- **판독성과 작성력**: 읽고 쓰기 쉬울수록 맞게 짤 가능성이 높다.

함정: "별칭"은 신뢰성을 *낮추는* 요인. 별칭 = 같은 메모리 셀에 대한 둘 이상의 참조 방법.

§ 비용 (Cost)

1. 프로그래머 교육 비용
2. 프로그램 작성 비용
3. 컴파일 비용
4. 실행 비용 (최적화 정도와 trade-off)
5. 언어 구현 시스템 비용
6. 신뢰성 부족에 따른 비용 (예: 핵발전소, Mars Pathfinder 우선순위 역전)
7. **유지보수 비용** (수정 + 기능 추가)

§ 기타 기준

- **이식성** (Portability): 한 구현에서 다른 구현으로 옮기기 쉬움. 표준화 정도가 가장 큰 영향.
- **일반성** (Generality): 넓은 범위의 응용에 적용 가능.
- **분명성** (Well-definedness): 공식 정의의 완전성과 정밀성.

설계에 대한 영향 · 언어 범주 · 설계 절충

교안 범위: L02 p.19–27

§ 언어 설계에 영향을 준 두 요인

1. **컴퓨터 구조** (폰 노이만 구조) → **명령형 언어**
2. **프로그래밍 설계 방법론** → 새 패러다임

폰 노이만 구조: 데이터와 프로그램이 같은 메모리에, 메모리는 CPU와 분리, 명령어·데이터가 메모리→CPU로 파이프됨. 명령형 언어와의 대응:

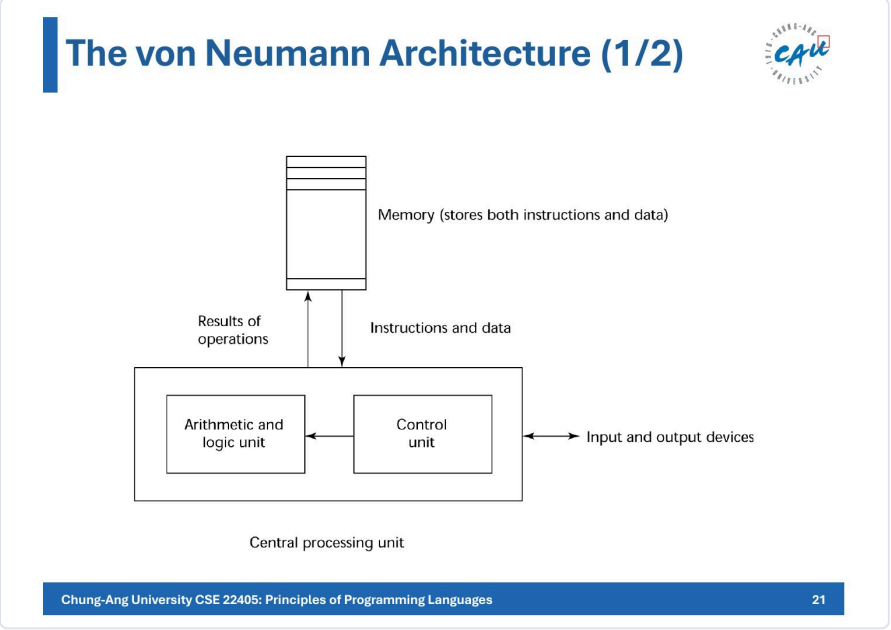
- **변수** ↔ 메모리 셀
- **배정문** ↔ 파이핑(데이터 이동)
- **반복** 이 효율적 (명령어가 인접 메모리에 있으므로)

폰 노이만 병목: 메모리↔CPU 버스 전송 속도가 명령 실행 속도보다 느려서 컴퓨터 속도의 주된 제한 요인이 됨.

Fetch-execute cycle:

```
initialize the program counter
repeat forever
    fetch the instruction pointed by the counter
    increment the counter
    decode the instruction
    execute the instruction
end repeat
```

방법론 변화: 1950s~60s 초 기계 효율 → 60s 후반 사람 효율(구조적 프로그래밍, 하향식 설계, 단계적 세분화) → 70s 후반 절차 지향→**데이터 지향**(데이터 추상화) → 80s 중반 **객체 지향**(데이터 추상화 + 상속 + 다형성/동적 바인딩).



L02 p.21 · 폰 노이만 구조: 메모리와 CPU가 분리, 명령어·데이터가 버스로 이동

§ 언어 범주

범주	핵심	예
명령형 (Imperative)	변수, 배정문, 반복. OOP·스크립트·비주얼 언어 포함	C, Java, Perl, JavaScript, C++
함수형 (Functional)	주어진 매개변수에 함수를 적용해 계산. **불변성, 순수 함수**	LISP, Scheme, ML, F#
논리형 (Logic)	규칙 기반, 규칙에 특정 순서가 없음	Prolog
마크업/프로그래밍 혼합	마크업 언어에 프로그래밍 기능 일부 추가	JSTL, XSLT

함정: 객체지향 언어와 스크립트 언어는 별도 범주가 아니라 **명령형에 포함**.

§ 언어 설계 절충 (Trade-offs)

"상충하는 기준을 절충하는 것이 주요 공학 과제" (Hoare). 교안의 3쌍과 예:

상충	예
신뢰성 vs 실행 비용	Java는 모든 배열 참조의 인덱스 범위를 검사 → 신뢰성↑ 실행 비용↑
판독성 vs 작성력	APL은 강력한 연산자·새 기호가 많아 짧게 쓰지만(작성력↑) 읽기 어려움(판독성↓)
작성력(유연성) vs 신뢰성	C++ 포인터는 강력·유연하지만 신뢰성이 낮음

시험 포인트: 24 중간 2번 = "세 쌍 중 하나 골라 간단한 예". 위 표의 예를 그대로 쓰면 된다. 쌍과 예를 섞어 외우지 마 (APL = 판독/작성, Java 배열 = 신뢰/비용, C++ 포인터 = 작성/신뢰).

수업 메모 02주-2
평가 기준들 사이에서 언어 설계자가 어떤 판단을 했는지를 보는 것이 절충. Java 배열 인덱스 검사는 실행 시간에 하므로 신뢰성은 올라가지만 실행 비용이 든다.

L1-5 · CH.1 기본사항 (PRELIMINARIES)

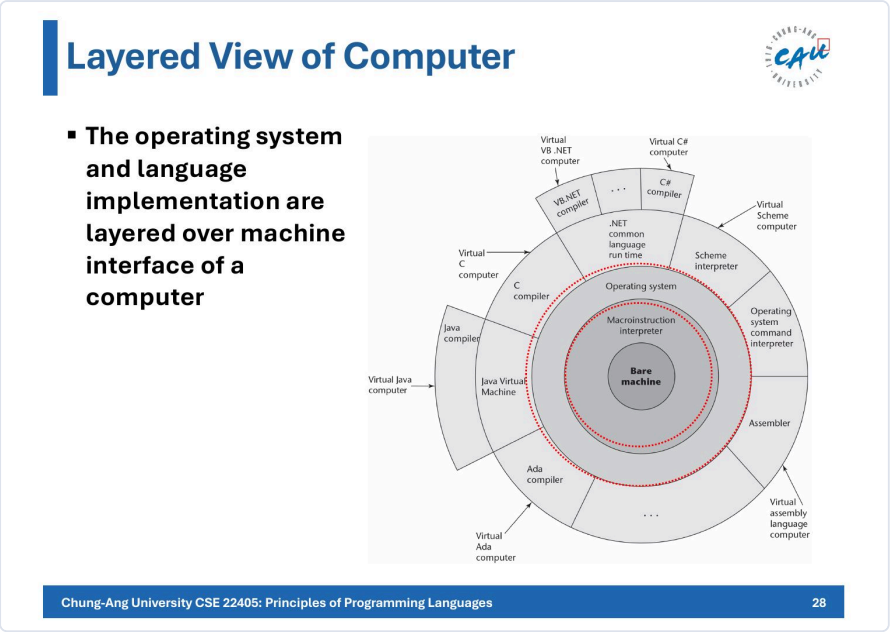
구현 방법 · 전처리기 · 프로그래밍 환경

교안 범위: L02 p.28–40

§ 구현 방법 3가지

방법	핵심	속도	용도
컴파일 (Compilation)	고급 언어 → 기계어로 번역 (JIT 포함)	번역 느림, **실행 빠름**	대형 상용 응용
순수 해석 (Pure interpretation)	번역 없이 인터프리터가 직접 해석·실행	실행 10~100배 느림, 공간 더 필요	작은 프로그램, 효율이 중요치 않을 때
혼합형 (Hybrid)	중간 언어로 번역 후 그것을 해석	순수 해석보다 빠름	효율이 최우선이 아닌 중소 시스템

> 시험 포인트: 24 중간 3번 = "세 가지 방법과 각각 간략히". 위 표의 "핵심" + 속도 특징 한 마디.



L02 p.28 · 컴퓨터의 계층적 관점: OS와 언어 구현 시스템이 기계 위에 층으로 쌓임

§ 컴파일 과정

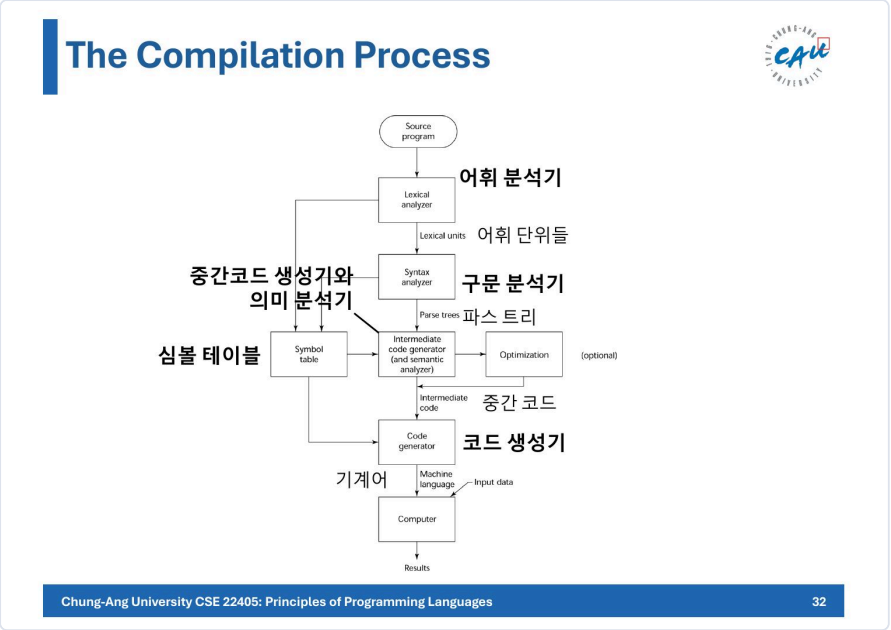
```
원시 프로그램 → [어휘 분석기] → 어휘 단위들
              → [구문 분석기] → 파스 트리
              → [중간 코드 생성기 + 의미 분석기] → 중간 코드
              → (최적화) → [코드 생성기] → 기계어
   (모든 단계가 심볼 테이블을 참조)
```

- **어휘 분석** (lexical analysis): 문자 → 어휘 단위(lexical units)
- **구문 분석** (syntax analysis): 어휘 단위 → 파스 트리 (구문 구조)
- **의미 분석**: 중간 코드 생성 (타입 오류 등 검사)
- **코드 생성**: 기계어 생성

용어:

- **링킹과 적재** (linking and loading): 시스템 프로그램(라이브러리)을 모아 사용자 프로그램(목적 코드)에 연결하는 과정.
- **적재 모듈** (load module, executable image): 링킹 결과물로 실행을 위해 메모리에 적재되는 것.

> **수업 메모** 02주-2 · 05주-1
> 의미 분석의 예: 문법(구문)은 맞지만 대입하는 양쪽 타입이 안 맞는 경우를 여기서 잡는다. 이 수업에서 다루는 핵심은 가운데 부분(어휘·구문·의미 분석). 실제 컴파일러는 파스 트리에 정보를 더한 AST를 만들지만 수업에선 파스 트리까지만.



L02 p.32 · 컴파일 과정 (어휘 분석기 → 구문 분석기 → 중간 코드 생성기/의미 분석기 → 코드 생성기)

§ 순수 해석 · 혼합형 · JIT

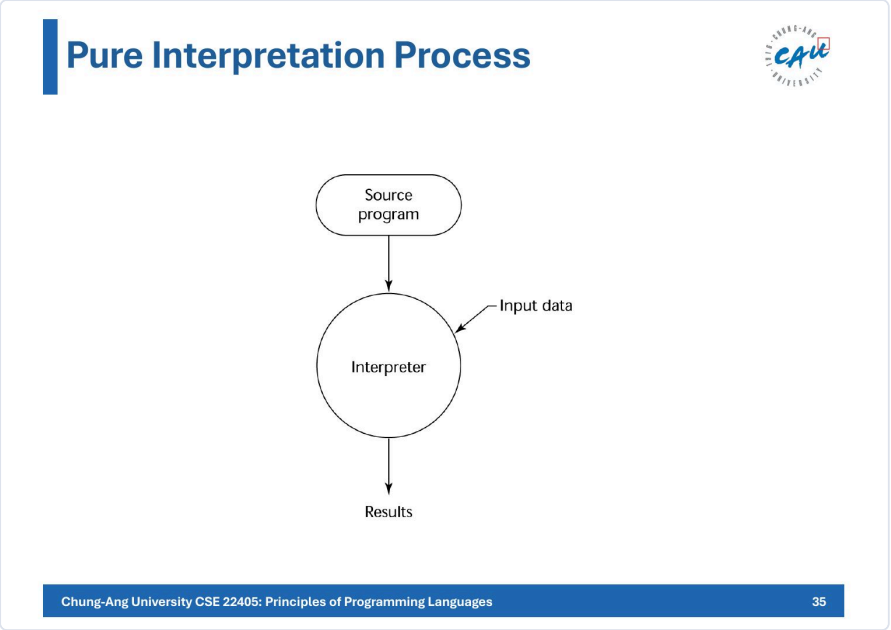
순수 해석: 번역 없음. 실행 시간 오류를 즉시 소스 수준에서 보여주기 쉬움. 느리고 공간이 더 필요. 전통 고급 언어에선 드물지만 웹 스크립트(JavaScript, PHP)로 부활.

혼합형: 고급 언어 → 해석하기 쉬운 **중간 언어**로 번역 → 해석.

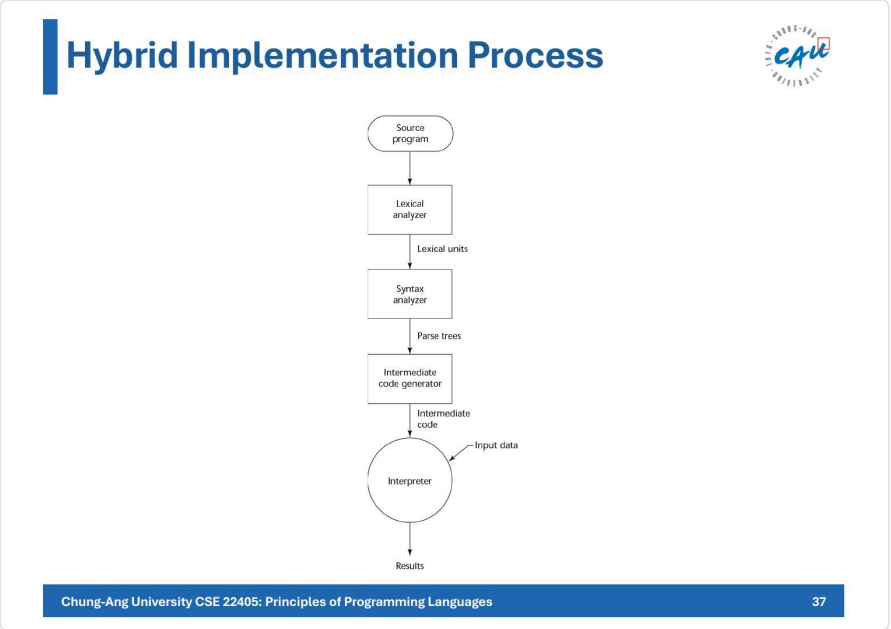
- Perl: 해석 전에 부분 컴파일해 오류 탐지.
- 초기 Java: 중간 형태 = **바이트 코드**. 바이트코드 인터프리터 + 런타임 시스템 = **JVM** → 이식성.

JIT (Just-in-Time): 먼저 중간 언어로 번역 → 부프로그램이 **호출될 때** 그 중간 코드를 기계어로 컴파일 → 기계어 버전은 이후 호출을 위해 보관. Java, .NET이 사용. 본질적으로 **지연된 컴파일러** (delayed compiler).

> **수업 메모** 02주-2
> 순수 해석은 한 줄씩 실행하니까 에러가 난 줄 번호·변수 이름을 바로 알려주기 쉽다 (컴파일된 기계어는 변수 이름을 기억하지 않음). 대신 실행 중에 변수 이름·타입 정보를 계속 들고 있어야 해서 공간이 더 필요하다. JIT: 중간 코드를 두고, 호출될 때 그 부분을 기계어로 컴파일해서 보관.



L02 p.35 · 순수 해석 과정



L02 p.37 · 혼합형 구현 과정 (중간 코드 → 인터프리터)

§ 전처리기와 프로그래밍 환경

전처리기 (preprocessor): 컴파일 직전에 프로그램을 처리해 내장된 전처리기 매크로를 확장.

- `#include "myLib.h"` → 그 위치에 파일 내용을 복사
- `#define max(A, B) ((A) > (B) ? (A) : (B))`

프로그래밍 환경: SW 개발 도구 모음 (언어 처리기, 편집기, 디버거, 빌드 도구, 라이브러리, IDE). 예: UNIX, Microsoft Visual Studio.NET, NetBeans(Java).

U2 · L03 Describing Syntax and Semantics.pdf (p.1-30)

Ch.3 구문 기술 (BNF · 파스 트리 · 모호성)

시험에서 손으로 그려야 하는 파트. 최좌단 유도, 파스 트리, 모호성 증명은 "읽기"보다 "직접 써보기"로 익혀. 퀴즈에서 실제로 유도를 쓰게 할 거야.

구문 vs 의미론 · 기본 용어

교안 범위: L03 p.3–8

§ 구문과 의미론

- **구문** (syntax): 식(expression), 문장(statement), 프로그램 단위(program unit)의 **형식 또는 구조**.
- **의미론** (semantics): 그 식·문장·프로그램 단위의 **의미**.

예) Java `while (bool_expr) statement`

- 구문: 위와 같은 형태로 써야 한다.
- 의미: bool_expr의 현재 값이 참이면 내포된 문장을 실행하고, 거짓이면 while 다음 문장으로 제어가 넘어간다.

언어 정의(구문+의미)를 쓰는 사람: **다른 언어 설계자**(모호성·비일관성 발견), **구현자**(올바른 구현), **프로그래머**(올바르게 사용).

> 시험 포인트: 24 중간 4번 "구문과 의미론을 간략히". 형식/구조 vs 의미, 그리고 while 예 하나면 충분.

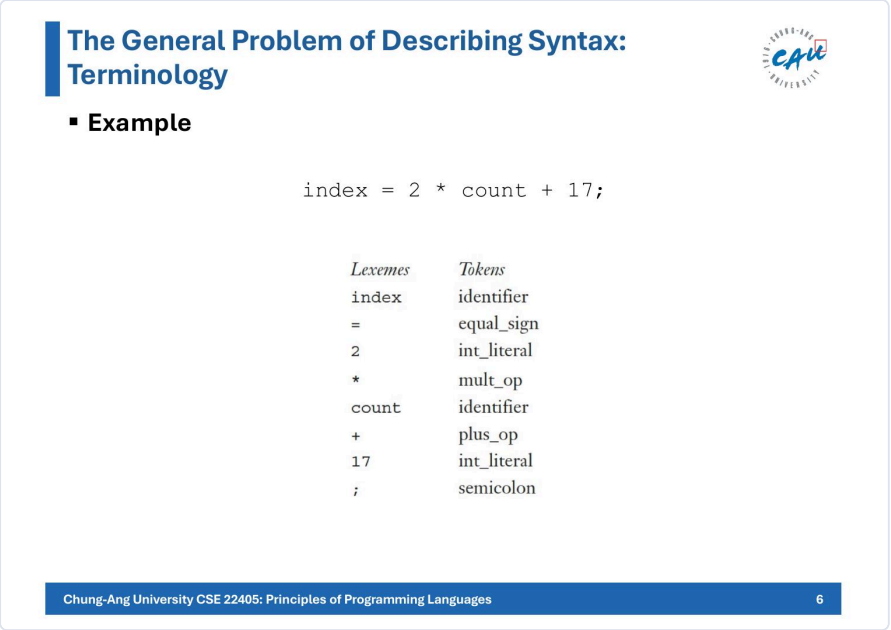
§ 문장 · 언어 · 어휘항목 · 토큰

- **문장** (sentence): 어떤 알파벳(Σ) 위의 문자열
- **언어** (language): 문장들의 집합
- **어휘항목** (lexeme): 언어의 **가장 낮은 수준의 구문 단위**. 예) `*`, `sum`, `begin`
- **토큰** (token): **어휘항목들의 범주(category)**. 예) identifier

`index = 2 * count + 17;` 를 쪼개면:

어휘항목 (lexeme)	토큰 (token)
index	identifier
=	equal_sign
2	int_literal
*	mult_op
count	identifier
+	plus_op
17	int_literal
;	semicolon

> 시험 포인트: 24 중간 6번. "어휘항목 = 가장 낮은 수준의 구문 단위(실제 문자열), 토큰 = 어휘항목의 범주(종류)". `index`와 `count`는 다른 lexeme인데 같은 token.



L03 p.6 · index = 2 * count + 17; 의 어휘항목과 토큰

§ 인식기와 생성기

언어를 형식적으로 정의하는 두 방법:

- **언어 인식기** (recognizer): 입력 문자열을 읽고 그 언어에 속하는지 판단하는 장치. 예) 컴파일러의 **구문 분석기** (Ch.4).
- **언어 생성기** (generator): 언어의 문장을 생성하는 장치. 어떤 문장이 구문적으로 맞는지를 생성기의 구조와 비교해 판단. (BNF 문법이 생성기 역할)

BNF와 유도

교안 범위: L03 p.8–15

§ BNF와 문맥 자유 문법

- **문맥 자유 문법 (CFG)**: Noam Chomsky, 1950년대 중반. 자연어 구문 기술용 언어 생성기. 문맥 자유 언어 클래스를 정의.
- **BNF** (Backus-Naur Form): John Backus, 1959, ALGOL 58 기술용. **BNF는 CFG와 동등**하다.
- **메타언어** (metalanguage): 다른 언어를 기술하는 언어. BNF는 프로그래밍 언어를 위한 메타언어.

BNF 구성 요소:

- **추상화 = 비단말 기호** (nonterminal): 구문 구조의 클래스, `<assign>`처럼 꺾쇠로 감쌈
- **단말 기호** (terminal): 어휘항목 또는 토큰
- **규칙 / 생성 규칙** (rule, production): `LHS → RHS`. LHS는 비단말 하나, RHS는 단말·비단말 문자열
- 한 비단말이 여러 RHS를 가지면 `|`로 묶음
- **문법** (grammar): 규칙들의 유한·비어있지 않은 집합
- **시작 기호** (start symbol): 유도를 시작하는 특별한 비단말

```
<if_stmt> → if (<logic_expr>) <stmt>
          | if (<logic_expr>) <stmt> else <stmt>
```

리스트는 재귀로 표현: LHS가 RHS에 다시 나오면 재귀 규칙.

```
<ident_list> → ident
             | ident , <ident_list>
```

§ 유도 · 문장 형태 · 최좌단 유도

- **유도** (derivation): 시작 기호에서 출발해 규칙을 반복 적용, 단말로만 된 문장에서 끝남.
- **문장 형태** (sentential form): 유도 중에 나오는 모든 기호열.
- **문장** (sentence): 단말 기호만 있는 문장 형태.
- **최좌단 유도** (leftmost derivation): 매 단계에서 **가장 왼쪽 비단말**을 전개.
- 유도 순서(최좌단/최우단/그 외)는 **생성되는 언어에 영향 없음**.

예제 문법:

```
<program> → begin <stmts> end
<stmts>   → <stmt> | <stmt> ; <stmts>
<stmt>    → <var> = <expr>
<var>     → a | b | c | d
<expr>    → <var> + <var> | <var> - <var> | <var>
```

`begin a = b + c; b = c end`의 최좌단 유도:

```
<program> ⇒ begin <stmts> end
          ⇒ begin <stmt> ; <stmts> end
          ⇒ begin <var> = <expr> ; <stmts> end
          ⇒ begin a = <expr> ; <stmts> end
          ⇒ begin a = <var> + <var> ; <stmts> end
          ⇒ begin a = b + <var> ; <stmts> end
          ⇒ begin a = b + c ; <stmts> end
          ⇒ begin a = b + c ; <stmt> end
          ⇒ begin a = b + c ; <var> = <expr> end
          ⇒ begin a = b + c ; b = <expr> end
          ⇒ begin a = b + c ; b = <var> end
          ⇒ begin a = b + c ; b = c end
```

쓰는 요령: 매 줄마다 "지금 가장 왼쪽 비단말이 뭐지?"만 보고 그거 하나만 바꿔. 두 개 동시에 바꾸면 감점.

수업 메모 03주-1

최좌단으로 유도하든 최우단으로 유도하든 마지막에 나오는 문장은 같다(유도 순서는 언어에 영향 없음). 시험에서 '최좌단 유도'라고 하면 그 순서를 지켜서 써야 한다.

파스 트리 · 모호성

교안 범위: L03 p.16–22

§ 파스 트리

파스 트리: 문장이 시작 기호에서 어떻게 유도되는지를 계층 구조(트리)로 보여줌.

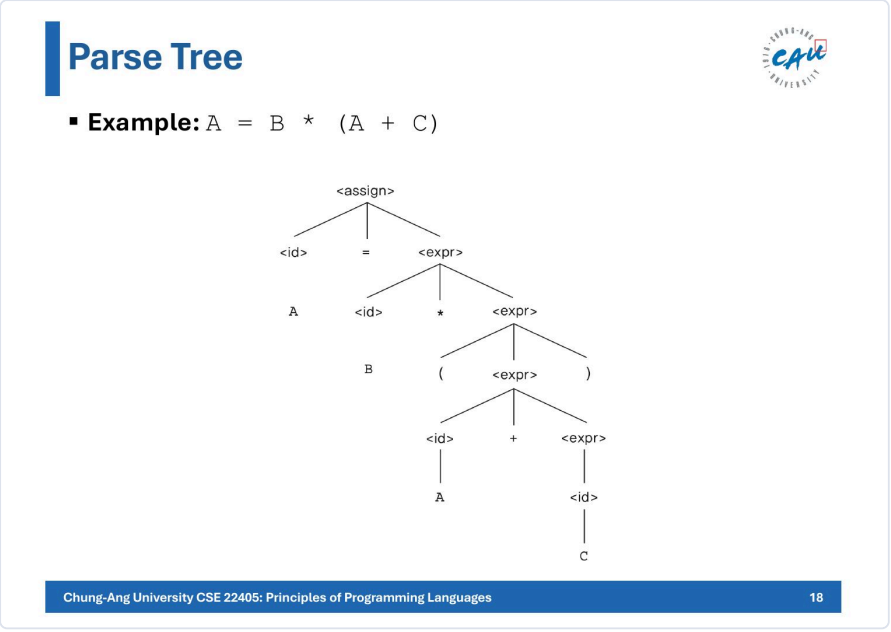
- 내부 노드 = 비단말, 잎 = 단말, 루트 = 시작 기호
- 노드의 자식들 = 그 노드에 적용한 규칙의 RHS (왼→오 순서)

문법 (교안 p.17):

```
<assign> → <id> = <expr>
<id>     → A | B | C
<expr>   → <id> + <expr> | <id> * <expr> | ( <expr> ) | <id>
```

`A = B * (A + C)` 의 파스 트리:

```
            <assign>
          /     |      \
       <id>     =      <expr>
        |             /   |    \
        A          <id>   *    <expr>
                    |         /   |   \
                    B        (  <expr>  )
                             /   |   \
                          <id>   +   <expr>
                           |           |
                           A         <id>
                                       |
                                       C
```



L03 p.18 · A = B * (A + C) 의 파스 트리

§ 모호성 (Ambiguity)

문법이 모호하다 ⇔ 어떤 문장 형태에 대해 서로 다른 파스 트리가 2개 이상 존재.
동치 조건: 어떤 문장에 대해 **최좌단 유도가 2개 이상** (또는 최우단 유도가 2개 이상).

모호한 문법:

```
<assign> → <id> = <expr>
<id>     → A | B | C
<expr>   → <expr> + <expr> | <expr> * <expr> | ( <expr> ) | <id>
```

`A = B + C * A` 에 대해 두 트리가 생긴다:

```
(1) + 가 위:  <expr> ⇒ <expr> + <expr>, 오른쪽 <expr> ⇒ <expr> * <expr>    → B + (C * A)
(2) * 가 위:  <expr> ⇒ <expr> * <expr>, 왼쪽  <expr> ⇒ <expr> + <expr>    → (B + C) * A
```

모호성 증명하는 법 (시험 답안 형식):

1. 문장 하나를 고른다 (짧을수록 좋음).
2. 그 문장에 대한 **서로 다른 최좌단 유도 2개** (또는 파스 트리 2개)를 쓴다.
3. "같은 문장에 대해 서로 다른 파스 트리가 2개 존재하므로 모호하다"로 마무리.

예) `<S> → <A>`, `<A> → <A> + <A> | <id>`, `<id> → a | b | c` 에서 `a + b + c`:

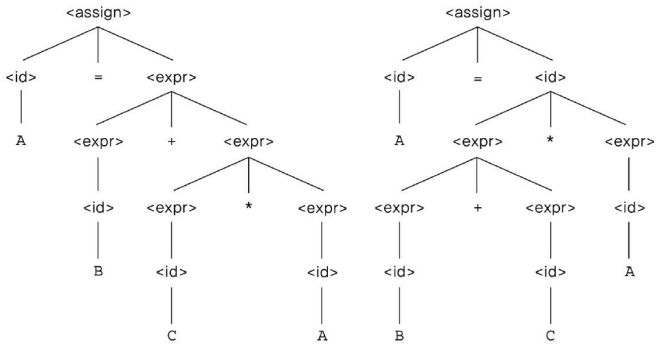
```
유도 1: <S> ⇒ <A> ⇒ <A> + <A> ⇒ <A> + <A> + <A> ⇒ <id> + <A> + <A>
        ⇒ a + <A> + <A> ⇒ a + <id> + <A> ⇒ a + b + <A> ⇒ a + b + <id> ⇒ a + b + c
        (첫 <A> + <A> 중 "왼쪽" <A>를 다시 <A> + <A>로 전개 → (a+b)+c)
유도 2: <S> ⇒ <A> ⇒ <A> + <A> ⇒ <id> + <A> ⇒ a + <A> ⇒ a + <A> + <A>
        ⇒ a + <id> + <A> ⇒ a + b + <A> ⇒ a + b + <id> ⇒ a + b + c
        (오른쪽 <A>를 전개 → a+(b+c))
```

시험 포인트: 24 중간 8번이 이 유형(3점). "두 개의 서로 다른 최좌단 유도 또는 파스 트리"를 반드시 보여야 함. 말로만 "두 가지로 해석된다"는 감점.

수업 메모 03주-1
모호성 = 같은 문장에 파스 트리가 2개 이상. 모호하면 컴파일러가 하나로 결정(deterministic)할 수 없어서 문제가 된다.

An Ambiguous Expression Grammar

- **Example:** `A = B + C * A`



Chung-Ang University CSE 22405: Principles of Programming Languages 22

L03 p.22 · A = B + C * A 의 서로 다른 두 파스 트리 (오른쪽 트리 2층의 <id>는 교안 오타, <expr>가 맞음)

우선순위 · 결합성 · if-else 모호성

교안 범위: L03 p.23–27

§ 연산자 우선순위를 문법으로

연산자마다 **별도의 비단말**을 두면 모호성이 사라진다.
규칙: **우선순위가 높은 연산자일수록 파스 트리에서 더 낮은 곳(잎 쪽)** 에 나타나야 한다.

모호하지 않은 식 문법 (**시험에 자주 주어지는 문법**):

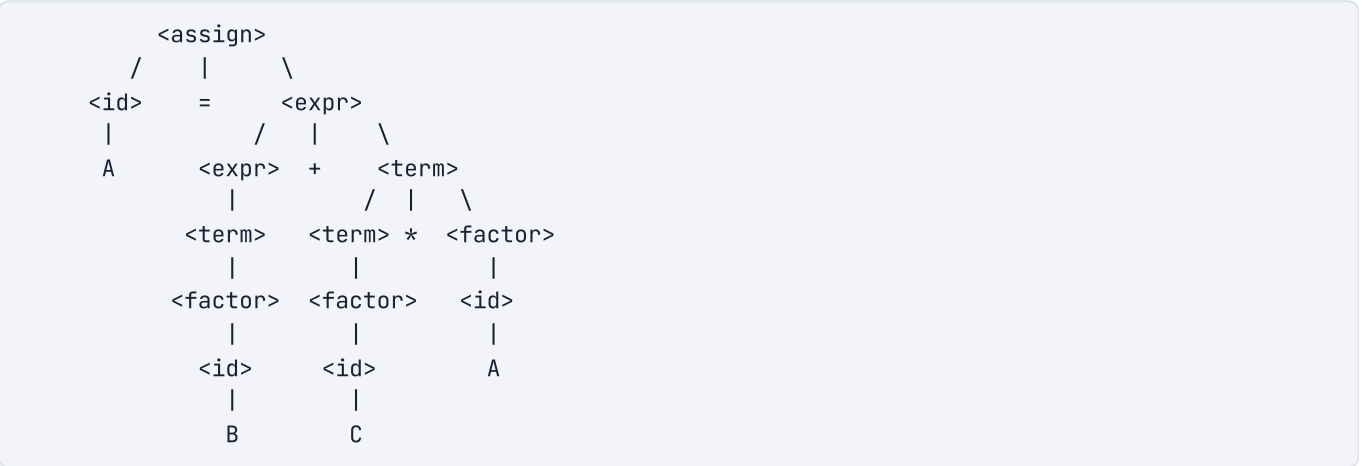
```
<assign> → <id> = <expr>
<id>     → A | B | C
<expr>   → <expr> + <term>   | <term>
<term>   → <term> * <factor> | <factor>
<factor> → ( <expr> )        | <id>
```

`*`는 `<term>` 단계, `+`는 `<expr>` 단계 → `*`가 항상 아래 → 먼저 계산.

`A = B + C * A` 의 최좌단 유도 (L04 p.37에도 나옴):

```
<assign> ⇒ <id> = <expr>
         ⇒ A = <expr>
         ⇒ A = <expr> + <term>
         ⇒ A = <term> + <term>
         ⇒ A = <factor> + <term>
         ⇒ A = <id> + <term>
         ⇒ A = B + <term>
         ⇒ A = B + <term> * <factor>
         ⇒ A = B + <factor> * <factor>
         ⇒ A = B + <id> * <factor>
         ⇒ A = B + C * <factor>
         ⇒ A = B + C * <id>
         ⇒ A = B + C * A
```

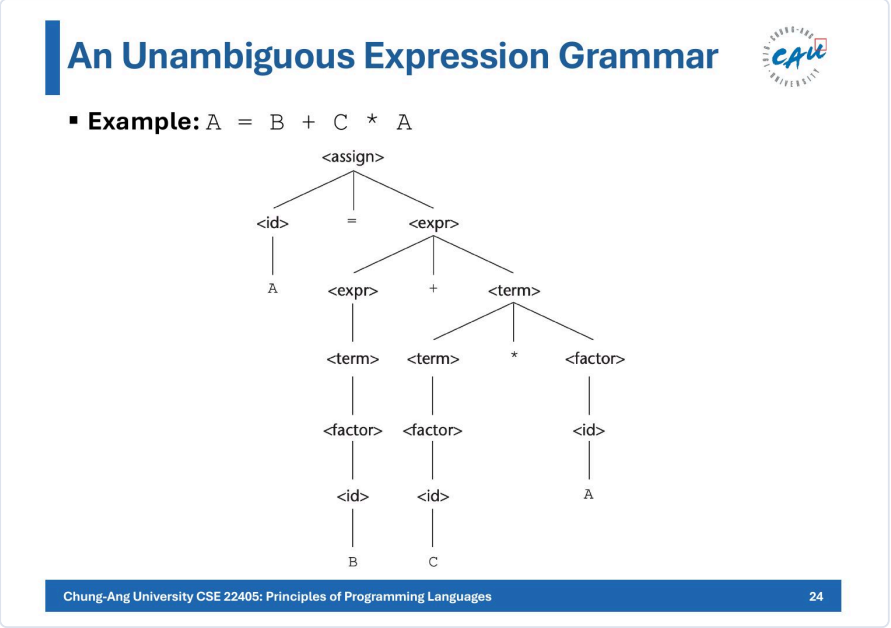
파스 트리:



괄호가 있으면 `<factor> → ( <expr> )` 를 써서 다시 `<expr>`로 올라간다. 24 중간 10번이 이 문법으로 괄호 있는 문장 2개를 유도하는 문제였어.

수업 메모 03주-1

파스 트리는 잎에서부터 위로 평가된다. 그래서 먼저 계산돼야 하는(우선순위 높은) 연산자가 트리 아래쪽에 있어야 한다. 그래서 우선순위가 낮은 + 를 <expr>에서 먼저(위에서) 전개하고, * 는 <term>에서 전개한다.



L03 p.24 · 모호하지 않은 문법으로 만든 A = B + C * A 의 파스 트리 (*가 더 아래)

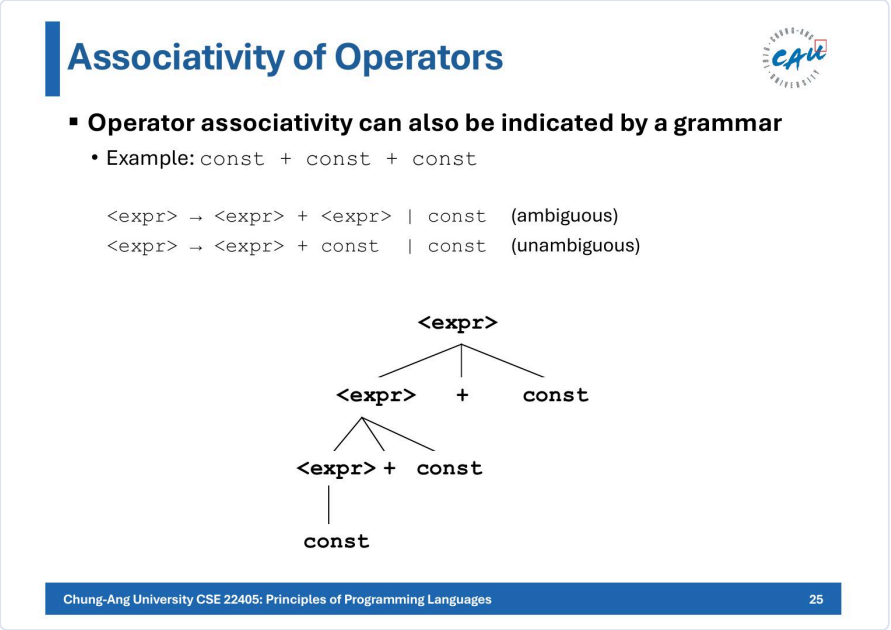
§ 결합성 (Associativity)

같은 우선순위 연산자가 연속될 때 어느 쪽부터 묶을지도 문법으로 표현한다.

```
<expr> → <expr> + <expr> | const    (모호)
<expr> → <expr> + const  | const    (모호하지 않음)
```

- **좌순환** 규칙 (LHS가 RHS의 **맨 왼쪽**에 나옴) → **좌결합**. `const + const + const` = `(const + const) + const`.
- **우순환** 규칙 (LHS가 RHS의 맨 오른쪽) → **우결합**. 예) 거듭제곱 `<factor> → <exp> ** <factor> | <exp>`.

함정: 좌순환 = 좌결합에 좋지만, Ch.4에서 배우는 **하향식 파서는 좌순환을 못 다룬다**. 이 연결고리가 시험에 나오기 좋다.



L03 p.25 · 좌순환 규칙 <expr> → <expr> + const 의 파스 트리 (왼쪽으로 깊어짐 = 좌결합)

§ if-then-else 모호성 (dangling else)

```
<if_stmt> → if (<logic_expr>) then <stmt>
          | if (<logic_expr>) then <stmt> else <stmt>
```

`if C1 then if C2 then S1 else S2` 에서 `else`가 바깥 if에 붙는지 안쪽 if에 붙는지 두 트리가 생김 → **모호**.

대부분의 언어 규칙: **else는 가장 가까운 짝 없는 then과 짝지어진다.**
모호하지 않은 문법:

```
<stmt>      → <matched> | <unmatched>
<matched>   → if (<logic_expr>) then <matched> else <matched>
            | a non-if statement
<unmatched> → if (<logic_expr>) then <stmt>
            | if (<logic_expr>) then <matched> else <unmatched>
```

핵심 아이디어: then과 else 사이에는 **matched**(else까지 짝이 다 맞은) 문장만 올 수 있다.

> **수업 메모** `03주-1`
> else는 매칭되지 않은 가장 가까운 then(if)과 짝지어진다는 규칙을 문법으로 표현한 게 matched/unmatched 문법.

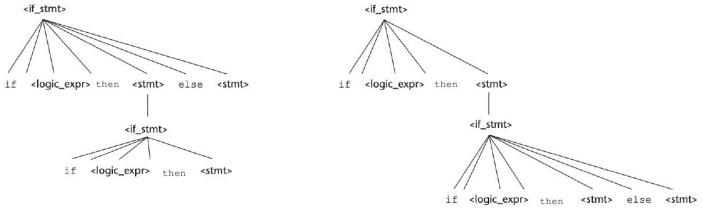
if-then-else (1/2)

- **Java** `if-then-else` **grammar**

```
<if_stmt> → if (<logic_expr>) then <stmt>
          | if (<logic_expr>) then <stmt> else <stmt>
```

Ambiguous!

Example: `if <logic_expr> then if <logic_expr> then <stmt> else <stmt>`



L03 p.26 · if-then-else 문법의 모호성: 같은 문장의 두 파스 트리

EBNF

교안 범위: L03 p.28–30

§ EBNF 확장 3가지

BNF를 짧게 쓰기 위한 확장. 표현력은 BNF와 같다.

표기	의미	예		
`[ ]`	선택(0 또는 1번)	`<if_stmt> → if (<expr>) <stmt> [else <stmt>]`		
`{ }`	0번 이상 반복	`<ident> → letter {letter`	digit}`	
`(  )`	여러 대안 중 하나	`<term> → <term> (+	-) const`	

BNF → EBNF 변환:

```
BNF:  <expr> → <expr> + <term> | <expr> - <term> | <term>
      <term> → <term> * <factor> | <term> / <factor> | <factor>
EBNF: <expr> → <term> {(+ | -) <term>}
      <term> → <factor> {(* | /) <factor>}
```

변환 요령: `A → A α | β` 꼴(좌순환)은 `A → β {α}` 로 바꾼다.

주의: EBNF로 바꾸면 좌순환이 `{}` 반복으로 사라지므로 결합성 정보가 문법에서 안 보인다 → 파서(구현)가 좌결합을 맞춰줘야 한다. 그리고 비단말 수가 줄어서 **재귀 하강 파서의 기반으로 딱 좋다** (Ch.4).

§ 최근의 EBNF 변형

- RHS 대안들을 각각 다른 줄에 씀, `→` 대신 `:` 사용
- 선택 부분에 아래첨자 `opt`: `ConstructorDeclarator → SimpleName (FormalParameterList_opt)`
- 선택지에 `one of`: `AssignmentOperator → one of = *= /= %= += -= ...`

U3 · L03 p.31–38 + 2026-02, PL, 03-02.pdf

Ch.3 속성 문법 · 의미론 (최약 전조건)

24 중간에서 9·11·12·19번, 합쳐서 23점이 공리 의미론이었어. 최약 전조건 계산은 기계적으로 풀리니까 여기서 점수를 확실히 챙겨.

정적 의미론 · 속성 문법

교안 범위: L03 p.31–38 (= 03-02 p.2–9)

§ 정적 의미론 (Static semantics)

CFG(BNF)로는 언어의 구문을 전부 기술할 수 없다. 문제가 되는 두 부류:

- 문맥 자유지만 **번거로운** 것: 식에서 피연산자의 타입 규칙 (모든 타입 조합을 규칙으로 쓰면 문법이 폭발)
- **문맥 자유가 아닌** 것: "변수는 사용 전에 선언돼야 한다"

정적 의미론:

- 이름과 달리 "의미"와는 거의 관계없음. 프로그램의 의미와 **간접적으로** 관련.
- 프로그램의 **적법한 형식** (legal forms)에 대한 규칙. 대부분 타입 제약.
- "정적" = **컴파일 시간**에 검사 가능.

> **수업 메모** 03주-2
> 예: A = A + 3 에서 A가 정수면 그냥 덧셈이지만 포인터면 주소 연산이 된다 → 타입 정보가 필요. '변수가 미리 선언됐는가'도 BNF만으로는 확인 못 한다. 이런 추가 정보를 붙이는 게 속성 문법.

§ 속성 문법의 정의

속성 문법 (AG) = CFG + 파스 트리 노드에 의미 정보(속성)를 붙이는 장치.
주된 용도: **정적 의미론 명세**, **컴파일러 설계**(정적 의미 검사).

CFG G = (S, N, T, P)에 다음을 추가:

- 각 문법 기호 X에 **속성 집합 A(X)**
- 각 규칙에 **의미 함수** (semantic function) 집합 + (비어 있을 수 있는) **술어 함수** (predicate function) 집합
- 술어 함수 = 속성들과 리터럴 값에 대한 **불리언 식**. 거짓이면 규칙 위반(오류).
- 모든 속성 값이 계산된 파스 트리를 **완전히 속성화된** (fully attributed) 트리라고 함.

§ 합성 · 상속 · 내장 속성

규칙 `X0 → X1 … Xn` 에 대해:

종류	형태	정보 흐름
합성 속성 (synthesized)	`S(X0) = f(A(X1), …, A(Xn))`	자식 → 부모 (**위로**)
상속 속성 (inherited)	`I(Xj) = f(A(X0), …, A(Xn))` , $1 \le j \le n$	부모·형제 → 자식 (**아래로/옆으로**)
내장 속성 (intrinsic)	잎(leaf) 노드에 처음부터 있는 값	예) 심볼 테이블에서 찾은 변수 타입

계산 순서:

- 모두 상속 속성이면 → **하향식** 순서로 트리를 장식할 수 있음
- 모두 합성 속성이면 → **상향식** 순서
- 보통은 둘 다 써서 하향식 + 상향식을 섞어야 함

외우는 법: **합성 = 아래에서 모아 올림(Synthesize), 상속 = 부모에게 물려받음(Inherit)**.

§ 속성 문법 예제

```
<assign> → <var> = <expr>
<expr>   → <var> + <var> | <var>
<var>    → A | B | C
```

- `actual_type` : `<var>`, `<expr>`의 **합성** 속성 (실제 타입)
- `expected_type` : `<expr>`의 **상속** 속성 (기대 타입, 부모가 정해줌)

규칙별 의미 함수 / 술어:

```
1. <assign> → <var> = <expr>
   의미 규칙: <expr>.expected_type ← <var>.actual_type
2. <expr> → <var>[1] + <var>[2]
   의미 규칙: <expr>.actual_type ← <var>[1].actual_type
   술어:     <var>[1].actual_type == <var>[2].actual_type
             <expr>.expected_type == <expr>.actual_type
3. <expr> → <var>
   의미 규칙: <expr>.actual_type ← <var>.actual_type
   술어:     <expr>.expected_type == <expr>.actual_type
4. <var> → A | B | C
   의미 규칙: <var>.actual_type ← lookup(<var>.string)
```

`A = A + B` 를 장식하는 순서:

1. `<var>.actual_type ← lookup(A)` (좌변 A)
2. `<expr>.expected_type ← <var>.actual_type` (상속: 위→아래)
3. `<var>[1].actual_type ← lookup(A)`, `<var>[2].actual_type ← lookup(B)` (내장/잎)
4. `<var>[1].actual_type =? <var>[2].actual_type` (술어)
5. `<expr>.actual_type ← <var>[1].actual_type` (합성: 아래→위)
6. `<expr>.actual_type =? <expr>.expected_type` (술어)

> **수업 메모** 03주-2
> 속성 값을 계산하는 순서 자체는 중요하지 않다. 어느 쪽부터 가든 결국 잎(터미널)의 내장 속성(심볼 테이블 lookup)까지 따라가서 값을 가져오면 된다.

동적 의미론 3가지 · 연산 · 표기 의미론

교안 범위: 03-02 p.10–19, 37

§ 왜 의미론을 형식적으로 기술하나

의미를 기술하는 단 하나의 표준 표기법은 없다. 그래도 필요한 이유:

- 프로그래머가 문장의 의미를 정확히 알아야 함
- 컴파일러 작성자가 구조가 정확히 뭘 하는지 알아야 함
- **정확성 증명**이 가능해짐 (공리 의미론)
- 컴파일러 생성기(compiler generator)가 가능해짐
- 설계자가 모호성·비일관성을 발견

§ 세 가지 방법 비교

방법	핵심 아이디어	기반
연산 의미론 (Operational)	이상적/가상 기계에서 실행했을 때 **기계 상태의 변화**로 의미를 정의	저수준 언어(가상 기계)
표기 의미론 (Denotational)	구문 요소를 **수학적 객체(재귀 함수)** 로 사상해 의미를 정의	재귀 함수 이론
공리 의미론 (Axiomatic)	각 문장에 **공리·추론 규칙(전조건/후조건)** 을 정의해 정확성을 증명	형식 논리(술어 계산)

시험 대비: "세 의미론을 각각 한 줄로" 문제를 대비해 위 표의 "핵심 아이디어"를 그대로 외워.

수업 메모 03주-2
세 가지 의미론 중 공리 의미론이 가장 중요하고 난이도도 있다고 강조 (계산 문제로 나옴). 연산·표기 의미론은 개념 위주로.

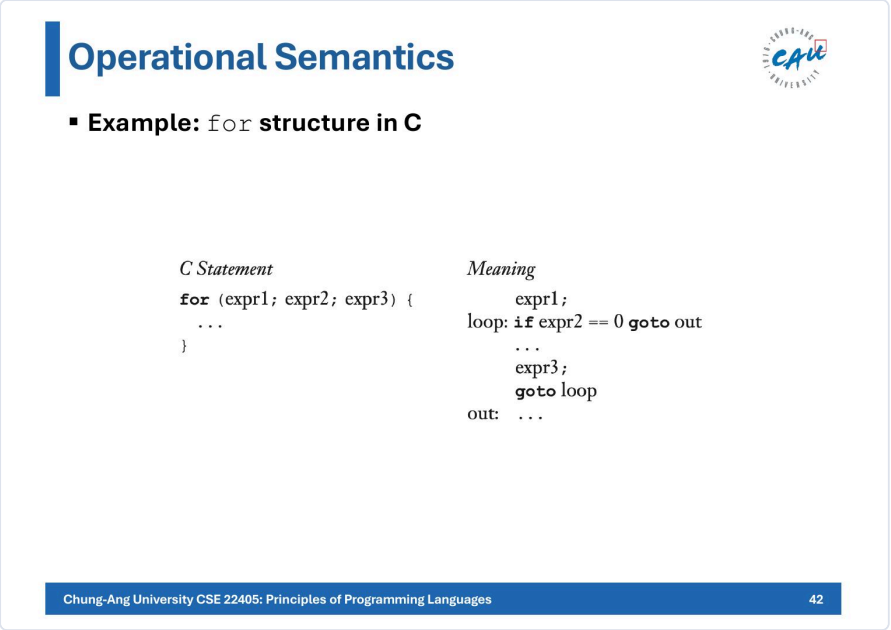
§ 연산 의미론

- 문장을 (시뮬레이션된 혹은 실제) 기계에서 실행해서 **기계 상태(메모리, 레지스터 등)의 변화**로 의미를 정의.
- 수학이 아니라 **더 낮은 수준의 프로그래밍 언어**의 문장으로 기술 → 고급 언어에 쓰려면 **가상 기계**가 필요.
- 예) C의 for문을 저수준 문장으로:

```
for (expr1; expr2; expr3) { ... }

        expr1;
loop:   if expr2 == 0 goto out
        ...
        expr3;
        goto loop
out:    ...
```

- 평가: 비형식적으로 쓰면 좋음(언어 매뉴얼 등). 형식적으로 쓰면 매우 복잡(VDL로 PL/I 기술).



03-02 p.13 · C의 for문을 저수준 문장으로 기술한 연산 의미론

§ 표기 의미론

- **재귀 함수 이론**에 기반.
- 명세 만드는 과정: (1) 각 언어 개체에 대응하는 **수학적 객체**를 정의 → (2) 언어 개체의 인스턴스를 수학적 객체의 인스턴스로 사상하는 **함수**를 정의.
  - 함수의 정의역 = **구문 정의역** (syntactic domain), 치역 = **의미 정의역** (semantic domain)
- 언어 구조의 의미는 **프로그램 변수들의 값만으로** 정의된다.
- **프로그램 상태**: `s = {<i1,v1>, <i2,v2>, …, <in,vn>}` (모든 현재 변수의 값)
- `VARMAP(ij, s) = vj` : 변수 이름과 상태를 받아 현재 값을 돌려줌

십진수 예:

```
<dec_num> → '0' | '1' | … | '9' | <dec_num> ('0' | '1' | … | '9')

Mdec('0') = 0, Mdec('1') = 1, …, Mdec('9') = 9
Mdec(<dec_num> '0') = 10 * Mdec(<dec_num>)
Mdec(<dec_num> '1') = 10 * Mdec(<dec_num>) + 1
…
Mdec(<dec_num> '9') = 10 * Mdec(<dec_num>) + 9
```

예) `Mdec('47') = 10 * Mdec('4') + 7 = 47`

논리 사전검사 루프 (상태 집합 → 상태 집합 U {error}):

```
Ml(while B do L, s) =
    if Mb(B, s) = undef then error
    else if Mb(B, s) = false then s
    else if Msl(L, s) = error then error
    else Ml(while B do L, Msl(L, s))
```

루프가 **재귀**로 표현된다는 게 핵심.

평가: 언어를 정밀하게 기술하는 훌륭한 방법, 언어 설계에 도움. 복잡해서 언어 사용자에게는 거의 쓸모없음.

연산 vs 표기: 연산 의미론은 상태 변화를 **코딩된 알고리즘**으로, 표기 의미론은 **엄밀한 수학 함수**로 정의.

공리 의미론 · 배정문의 최약 전조건

교안 범위: 03-02 p.20–26

§ 단언 · 전조건 · 후조건

- 공리 의미론은 **형식 논리(술어 계산)** 에 기반. 원래 목적: **형식적 프로그램 검증**.
- **단언** (assertion): 공리 의미론에서 쓰는 논리식(술어).
- **전조건** (precondition): 문장 **앞**의 단언. 그 실행 지점에서 참인 변수 간 관계·제약.
- **후조건** (postcondition): 문장 **뒤**의 단언.
- 표기: `{P} statement {Q}` (Hoare triple)
- **최약 전조건** (weakest precondition): 후조건을 보장하는 **가장 덜 제한적인** 전조건.

예) `a = b + 1 {a > 1}`

- 가능한 전조건: `{b > 10}` (맞지만 너무 강함)
- 최약 전조건: `{b > 0}`

표기: `wp(문장, 후조건) = 전조건`

증명 과정: 프로그램 전체의 후조건(원하는 결과)에서 시작해 **거꾸로** 첫 문장까지 전조건을 계산. 그 결과가 프로그램 명세(입력 조건)와 같으면(또는 명세가 그것을 함의하면) 프로그램은 정확하다.

§ 배정문 공리

`x = E` 의 후조건이 Q일 때:
전조건 P = Q에서 x를 모두 E로 치환한 것 (`P = Q_{x→E}`)

푸는 법은 딱 하나야: **후조건의 변수를 우변 식으로 바꿔 끼우고, 부등식을 정리.**

예 1) `a = b / 2 - 1 {a < 10}`

```
b / 2 - 1 < 10  →  b / 2 < 11  →  b < 22
wp = {b < 22}
```

예 2) `x = x + y - 3 {x > 10}`

```
x + y - 3 > 10  →  x + y > 13
wp = {x + y > 13}     ← 우변에 x 자신이 있어도 그냥 치환하면 됨
```

예 3) `sum = 2 * x + 1 {sum > 1}`

```
2x + 1 > 1  →  2x > 0  →  x > 0
```

실수 주의: (1) 치환은 후조건의 변수 **전부**에. (2) 괄호 분배 `2 * (b - 1)` = `2b - 2`. (3) 음수로 나누면 부등호 방향이 뒤집힘. 예) `-2x > 4` → `x < -2`.

§ 정리(theorem)로서의 배정문

전조건과 후조건이 **둘 다 주어진** 배정문은 하나의 논리 문장(정리)이다.
배정 공리를 후조건에 적용해서 **주어진 전조건이 나오면** 증명 끝.

예) `{x > 3} x = x - 3 {x > 0}`

```
wp = (x - 3 > 0) = {x > 3}  → 주어진 전조건과 같음 → 참(증명됨)
```

예) `{x > 5} x = x - 3 {x > 0}` : wp는 `{x > 3}`라서 배정 공리만으로는 안 나온다 → **결과 규칙**이 필요 (다음 레슨).

결과 규칙 · 시퀀스 · 선택문

교안 범위: 03-02 p.27–31

§ 추론 규칙과 결과 규칙

추론 규칙 (inference rule): 다른 단언들의 참을 근거로 한 단언의 참을 추론.

```
S1, S2, …, Sn      (위: 전제들)
──────────────
      S            (아래: 결론)
```

결과 규칙 (rule of consequence):

```
{P} S {Q},   P' ⇒ P,   Q ⇒ Q'
──────────────────────────────
         {P'} S {Q'}
```

뜻: 전조건은 **더 강하게**(P'⇒P), 후조건은 **더 약하게**(Q⇒Q') 바꿔도 여전히 성립.

예) `{x > 5} x = x - 3 {x > 0}` 증명:

```
{x > 3} x = x - 3 {x > 0}   (배정 공리)
(x > 5) ⇒ (x > 3)           (참)
──────────────────────────────
{x > 5} x = x - 3 {x > 0}
```

§ 시퀀스의 최약 전조건

```
{P1} S1 {P2},  {P2} S2 {P3}
──────────────────────────
     {P1} S1; S2 {P3}
```

푸는 법: 마지막 문장부터 거꾸로. S2의 wp를 구하고, 그걸 S1의 후조건으로 써서 S1의 wp를 구한다.

예) `y = 3 * x + 1; x = y + 3; {x < 10}`

```
1) x = y + 3 {x < 10}       → y + 3 < 10  → {y < 7}
2) y = 3 * x + 1 {y < 7}    → 3x + 1 < 7  → 3x < 6 → {x < 2}
wp = {x < 2}
```

실수 주의: **위에서부터 풀면 틀린다.** 항상 맨 아래 후조건 → 마지막 문장 → … → 첫 문장.

§ 선택문의 최약 전조건

```
{B and P} S1 {Q},   {(not B) and P} S2 {Q}
──────────────────────────────────────────
      {P} if B then S1 else S2 {Q}
```

푸는 법:

1. then 쪽: `wp(S1, Q)` 구하기
2. else 쪽: `wp(S2, Q)` 구하기

3. 두 전조건을 합친다.

예) `if x > 0 then y = y - 1 else y = y + 1 {y > 0}`

```
then: y - 1 > 0 → {y > 1}
else: y + 1 > 0 → {y > -1}
```

합치는 방법 두 가지 — 둘 다 알아둬:

- **교재(Sebesta)/교안 방식**: 두 전조건 모두를 만족시키는 하나를 고른다. `y > 1 ⇒ y > -1` 이므로 **`{y > 1}`**. (결과 규칙으로 else 쪽 전조건을 강화)
- **정밀한 최약 전조건**: `{(x > 0 and y > 1) or (x ⩽ 0 and y > -1)}`

> 시험 답안: 교재 방식 `{y > 1}`를 결론으로 쓰고, then/else 각각의 계산 과정을 꼭 보여. 24 중간 12번(5점)이 이 유형. a == b 같은 조건이 있으면 그 조건을 대입해 정리할 수도 있다는 걸 기억 (예: B가 `a == b` 면 then 쪽에서 b를 a로 볼 수 있음).

루프 불변자 · 정확성 증명

교안 범위: 03-02 p.32–36

§ 논리 사전검사 루프의 추론 규칙

```
     {I and B} S {I}
————————————————————————————
{I} while B do S end {I and (not B)}
```

I = 루프 불변자 (loop invariant). 귀납 가설 역할.

`{P} while B do S end {Q}` 를 증명하려면 I가 다음 4가지를 만족해야 함:

1. **P ⇒ I** : 루프 시작 전에 I가 참
2. **{I and B} S {I}** : 몸체를 한 번 실행해도 I가 유지됨
3. **(I and (not B)) ⇒ Q** : 루프가 끝났을 때(B 거짓) I가 참이면 Q가 함의됨
4. **루프가 종료함** : 증명이 어려울 수 있음

- 1~3만 증명 = **부분 정확성** (partial correctness)
- 4(종료)까지 증명 = **전체 정확성** (total correctness)

§ 루프 불변자 찾기

- 불변자 I는 루프 **후조건의 약화된 버전**이면서 **전조건**이기도 하다.
- 루프 시작 전에 만족될 만큼 **약해야** 하고, 종료 조건(not B)과 합쳐졌을 때 후조건을 강제할 만큼 **강해야** 한다.

예) `while y <> x do y = y + 1 end {y = x}`
반복 횟수 0, 1, 2...에 대해 wp를 구해 보면:

```
0회: {y = x}
1회: {y = x - 1}
2회: {y = x - 2}
…
→ 패턴: {y ⩽ x}  ← 이게 불변자 I이자 전조건 P
```

확인: `I and not B` = `y ⩽ x and y = x` ⇒ `y = x` ✓

§ 정확성 증명 답안 형식

예) 팩토리얼

```
{n ⩾ 0}
count = n;
fact = 1;
while count <> 0 do
    fact = fact * count;
    count = count - 1;
end
{fact = n!}
```

불변자: `I = (fact = (count+1) * (count+2) * … * n) and (count ⩾ 0)`
(count = n일 때 빈 곱 = 1)

1. **P ⇒ I**: 루프 직전 count = n, fact = 1. 빈 곱 = 1이므로 I 성립, count = n ≥ 0 ✓
2. **{I and B} S {I}**: count ≠ 0인 상태에서 `fact = fact * count` 하면 fact = count * (count+1) * … * n, 그 다음 `count = count - 1` 하면 fact = (count+1) * … * n 이 다시 성립. count ≥ 1 이었으므로 count ≥ 0 유지 ✓
3. **(I and not B) ⇒ Q**: count = 0이면 fact = 1 * 2 * … * n = n! ✓
4. **종료**: count는 n ≥ 0에서 시작해 매번 1씩 감소 → 언젠가 0 → 종료 ✓

> 시험 포인트: 24 중간 19번(10점)이 합(sum) 버전이었어. 답안 = ① 불변자 제시 ② 위 4가지를 하나씩 확인. 구조만 지키면 부분 점수가 크게 나온다.

---

§ 공리 의미론 평가

- 언어의 모든 문장에 대해 공리·추론 규칙을 만드는 것은 **어렵다**.
- **정확성 증명**에는 좋은 도구, 프로그램 추론에 훌륭한 틀.
- 하지만 언어 사용자와 컴파일러 작성자에게는 그다지 유용하지 않음.

U4 · L04 Lexical and Syntax Analysis.pdf (p.1-31) · 04_code.pptx

Ch.4 어휘 분석

어휘 분석기 = 패턴 매처. front.c가 (sum + 47) / total을 어떻게 토큰으로 자르는지 손으로 따라갈 수 있으면 끝.

구문 분석의 두 부분 · 분리하는 이유

교안 범위: L04 p.3–7

§ BNF 기반 구문 분석

언어 구현 시스템은 구현 방식(컴파일/해석/혼합)과 상관없이 **원시 코드를 분석**해야 하고, 거의 모든 구문 분석은 **BNF**에 기반한다.

BNF로 구문을 기술하는 장점:

- 명확하고 간결한 구문 기술
- 파서를 BNF에 **직접** 기반해서 만들 수 있음
- BNF 기반 파서는 유지보수가 쉬움 (모듈성)

§ 어휘 분석기 vs 구문 분석기

구문 분석 부분은 거의 항상 두 부분으로 나뉜다:

부분	역할	수학적 모델
어휘 분석기 (lexical analyzer)	저수준: 문자 → 어휘항목/토큰	**유한 오토마타** (정규 문법 기반)
구문 분석기 = 파서 (syntax analyzer, parser)	고수준: 토큰 → 파스 트리	**푸시다운 오토마타** (CFG/BNF 기반)

짝 외우기: 어휘 = 정규 문법 = 유한 오토마타 / 구문 = 문맥 자유 문법 = 푸시다운 오토마타.

수업 메모 05주-1
예전 컴파일러는 어휘 분석 결과를 토큰 파일로 출력하고 파서가 그 파일을 읽었다. 요즘은 파서가 토큰이 필요할 때 어휘 분석기 함수(lex)를 호출해서 하나씩 받아간다. 어휘 분석의 정규 문법 = 정규 표현식.

§ 어휘 분석과 구문 분석을 분리하는 이유

1. **단순성** (Simplicity): 어휘 분석엔 덜 복잡한 기법으로 충분. 분리하면 파서가 단순해진다(공백·주석 같은 세부를 파서가 신경 안 씀).
2. **효율성** (Efficiency): 어휘 분석이 컴파일 시간의 큰 부분이라, 분리하면 어휘 분석기만 따로 **최적화**할 수 있다.
3. **이식성** (Portability): 어휘 분석기는 입력 파일·버퍼링 등을 다뤄서 플랫폼 의존적일 수 있지만, **파서는 항상 이식 가능**.

시험 대비: "분리 이유 3가지" = 단순성·효율성·이식성. 각 한 줄.

수업 메모 05주-1
이식성 설명: 어휘 분석기는 파일 입출력·버퍼를 다뤄서 플랫폼에 의존적이다. 둘을 나눠두면 플랫폼이 바뀌어도 어휘 분석기만 고치면 되고 파서는 그대로 쓸 수 있다.

어휘 분석기 · 상태 다이어그램

교안 범위: L04 p.8–16

§ 어휘 분석기의 역할

- 문자열에 대한 **패턴 매처** (pattern matcher)
- 파서의 **전단부** (front-end)
- 원시 프로그램에서 같이 묶이는 부분 문자열(**어휘항목**)을 찾아내고, 그 어휘항목이 매칭된 패턴의 범주(**토큰**)를 정한다. 예) `sum`은 lexeme, 토큰은 `IDENT`.
- 보통 **파서가 다음 토큰이 필요할 때 호출하는 함수**로 구현된다.
- 부가 역할: 주석·공백 건너뛰기, 사용자 정의 이름을 심볼 테이블에 넣기, 어휘 오류 탐지.

`result = oldsum - value / 100;` →

토큰	어휘항목
IDENT	result
ASSIGN_OP	=
IDENT	oldsum
SUB_OP	-
IDENT	value
DIV_OP	/
INT_LIT	100
SEMICOLON	;

> **수업 메모** 05주-1
> 공백으로 어휘항목을 쪼개면 안 된다. 실제 코드에선 a+b 처럼 공백 없이도 쓰기 때문. 패턴(첫 글자가 숫자면 식별자가 될 수 없음 등)으로 판단해야 한다.

§ 어휘 분석기를 만드는 3가지 방법

1. 토큰을 **형식적으로 기술**(정규식)하고, 이 기술로부터 **표 기반** 어휘 분석기를 만들어주는 **SW 도구**를 사용 (예: lex)
2. 토큰을 기술하는 **상태 다이어그램**을 설계하고, 그 다이어그램을 **구현하는 프로그램**을 작성
3. 상태 다이어그램을 설계하고, 그것의 **표 기반 구현을 손으로** 작성

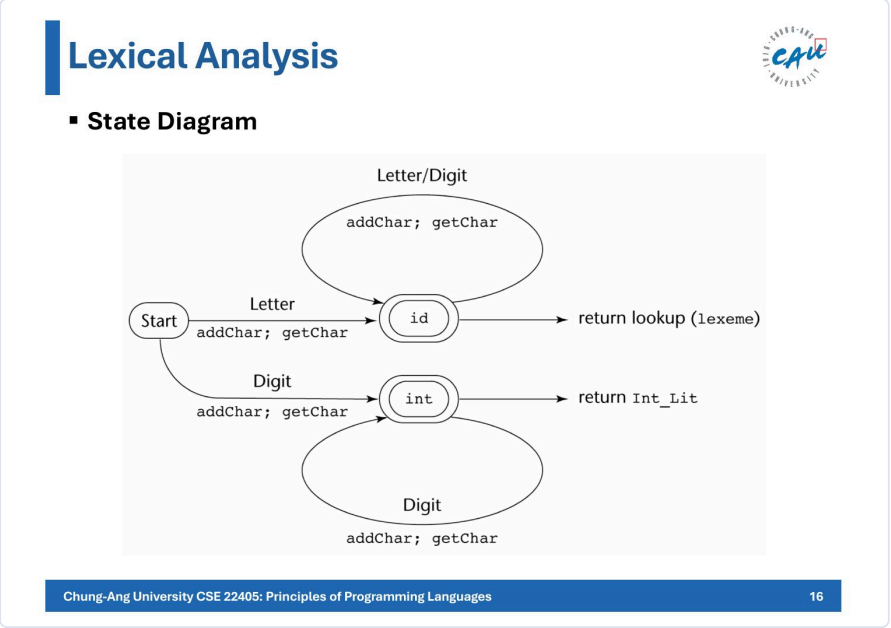
§ 상태 다이어그램 단순화

순진하게 그리면 모든 상태에서 모든 문자에 대한 전이가 있어야 해서 **너무 크다**. 단순화 방법:

- **문자 클래스**: 식별자 인식 중엔 대소문자 모든 글자가 동등 → `LETTER` 클래스 하나로. 정수 리터럴은 모든 숫자가 동등 → `DIGIT` 클래스.
- **예약어와 식별자를 함께 인식**: 예약어마다 다이어그램을 따로 그리지 않고, 식별자처럼 인식한 다음 **테이블 탐색 (lookup)** 으로 예약어인지 판별.

수업 메모 05주-1

예약어용 다이어그램을 따로 만들 필요 없이, 식별자로 인식한 뒤 예약어 테이블을 lookup해서 있으면 예약어, 없으면 식별자. 이 중 원은 종료(final) 상태.



L04 p.16 · 식별자·정수 리터럴을 인식하는 상태 다이어그램

§ 유틸리티 부프로그램

함수	하는 일
`getChar`	입력의 다음 문자를 `nextChar`에 넣고, 그 문자의 클래스를 판단해 `charClass`에 넣음
`addChar`	`nextChar`의 문자를 어휘항목을 모으는 곳(`lexeme`)에 추가
`lookup`	`lexeme`(또는 문자)가 예약어/연산자인지 판단해 토큰 코드를 돌려줌
`getNonBlank`	공백이 아닌 문자가 나올 때까지 `getChar` 호출

함정: 클래스를 정하는 건 `getChar`, 문자를 이어붙이는 건 `addChar`.

front.c 따라가기

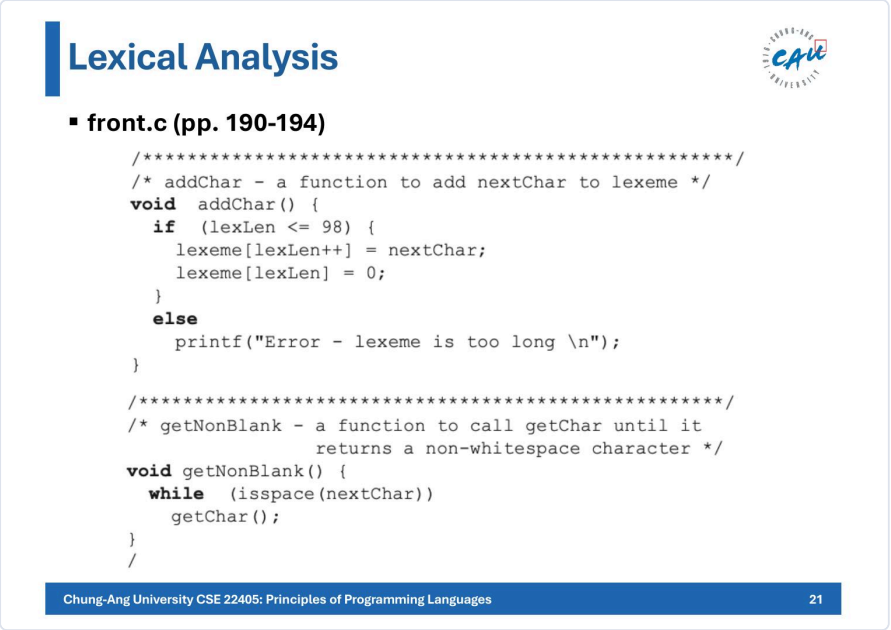
교안 범위: L04 p.17–31 · 04_code.pptx

§ front.c의 토큰 코드

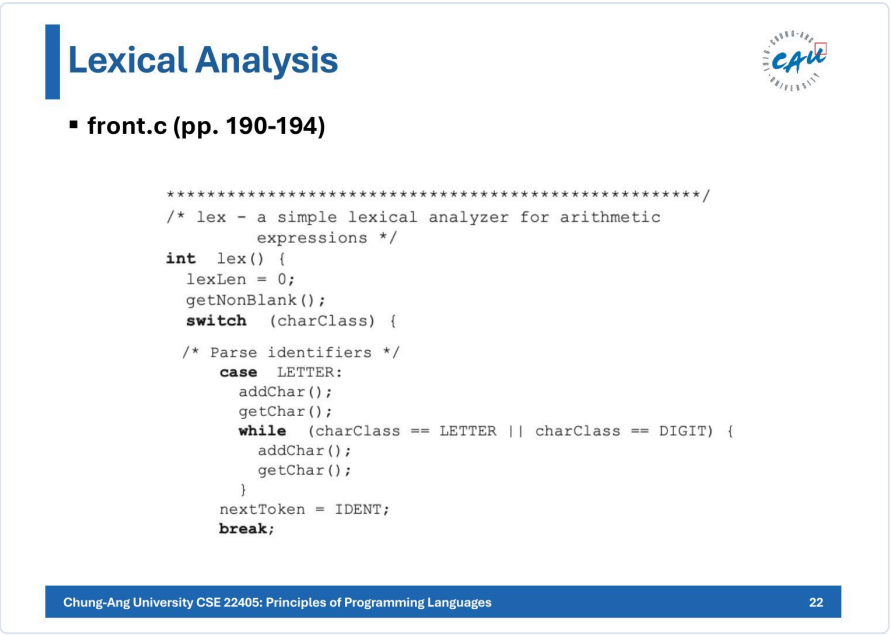
```
#define LETTER 0        #define INT_LIT 10       #define ADD_OP 21
#define DIGIT 1         #define IDENT 11         #define SUB_OP 22
#define UNKNOWN 99      #define ASSIGN_OP 20     #define MULT_OP 23
                                                 #define DIV_OP 24
                                                 #define LEFT_PAREN 25
                                                 #define RIGHT_PAREN 26
EOF = -1
```

`lex()` 의 구조:

```
int lex() {
    lexLen = 0;
    getNonBlank();
    switch (charClass) {
        case LETTER:                /* 식별자 */
            addChar(); getChar();
            while (charClass == LETTER || charClass == DIGIT) {
                addChar(); getChar();
            }
            nextToken = IDENT; break;
        case DIGIT:                 /* 정수 리터럴 */
            addChar(); getChar();
            while (charClass == DIGIT) { addChar(); getChar(); }
            nextToken = INT_LIT; break;
        case UNKNOWN:               /* 괄호와 연산자 */
            lookup(nextChar); getChar(); break;
        case EOF:
            nextToken = EOF; lexeme = "EOF"; break;
    }
    printf("Next token is: %d, Next lexeme is %s\n", nextToken, lexeme);
    return nextToken;
}
```



L04 p.21 · front.c: addChar, getChar



L04 p.22 · front.c: getNonBlank, lex

§ (sum + 47) / total 추적

출력:

```
Next token is: 25 Next lexeme is (
Next token is: 11 Next lexeme is sum
Next token is: 21 Next lexeme is +
Next token is: 10 Next lexeme is 47
Next token is: 26 Next lexeme is )
Next token is: 24 Next lexeme is /
Next token is: 11 Next lexeme is total
Next token is: -1 Next lexeme is EOF
```

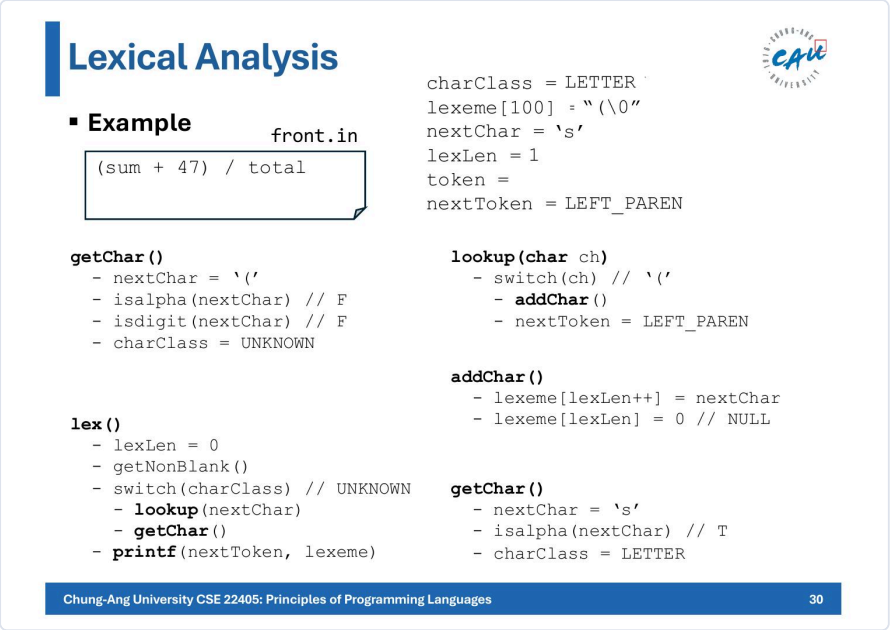
첫 두 토큰을 변수 단위로 따라가면 (04_code.pptx):

```
[처음 getChar()]  nextChar='('  isalpha F, isdigit F → charClass=UNKNOWN
[lex() #1] lexLen=0, getNonBlank()
           switch(UNKNOWN) → lookup('(') → addChar(): lexeme="(" , nextToken=LEFT_PAREN(25)
           getChar(): nextChar='s' → charClass=LETTER
           printf → "25 (",
[lex() #2] lexLen=0, getNonBlank()
           switch(LETTER) → addChar(): lexeme="s"; getChar(): nextChar='u' LETTER
           while: addChar() "su"; getChar() 'm' LETTER
                  addChar() "sum"; getChar() ' ' → UNKNOWN → while 종료
           nextToken=IDENT(11) → printf "11 sum"
[lex() #3] getNonBlank(): ' '는 공백 → getChar() → nextChar='+' UNKNOWN
           lookup('+') → ADD_OP(21) ...
```

포인트: `lex()`는 항상 **다음 문자를 하나 미리 읽어 둔 상태**(nextChar)로 끝난다. 그래서 식별자 끝을 알 수 있음(공백이나 연산자가 나오면 멈춤). 예약어 판별은 front.c에선 생략됨.

수업 메모 05주-1

과제 관련 공지: 과제는 C/C++로만. front.c처럼 입력 파일을 "front.in"으로 하드코딩하지 말고 main(int argc, char** argv)로 파일 이름을 받아서 처리할 것.



L04 p.30 · (sum + 47) / total 첫 토큰 처리 중 변수 상태

U5 · L04 p.32–62

Ch.4 파싱 문제 · 하향식 파싱

하향식 = 최좌단 유도 = 재귀 하강/LL. 시험 13번(집합쌍 공통 테스트)과 7번(하향식 vs 상향식)이 여기서 나와.

파싱 문제 · 하향식 vs 상향식

교안 범위: L04 p.32–42

§ 파서의 목표

파싱: 토큰 열이 언어의 문법에 맞는지 분석하는 과정.
파서의 두 목표:

1. **모든 구문 오류를 찾는다.** 각각에 적절한 진단 메시지를 내고 빨리 **복구**한다.
2. **파스 트리**(또는 최소한 파스 트리의 흔적)를 만든다.

§ 하향식 vs 상향식

	하향식 (Top-down)	상향식 (Bottom-up)
트리 생성	**루트**부터 잎 방향으로	**잎**부터 루트 방향으로
유도 순서	**최좌단 유도** 순서	**최우단 유도의 역순**
순회	전위 순회(preorder)로 추적/생성	
결정할 것	문장 형태 `xAα`에서 **다음 토큰 하나만 보고** 올바른 A-규칙 선택	우문장 형태에서 **어떤 부분 문자열(핸들)을 LHS로 환원**할지
대표 알고리즘	**재귀 하강**(코드 구현), **LL** 파서(표 기반)	**LR** 계열 (shift-reduce)

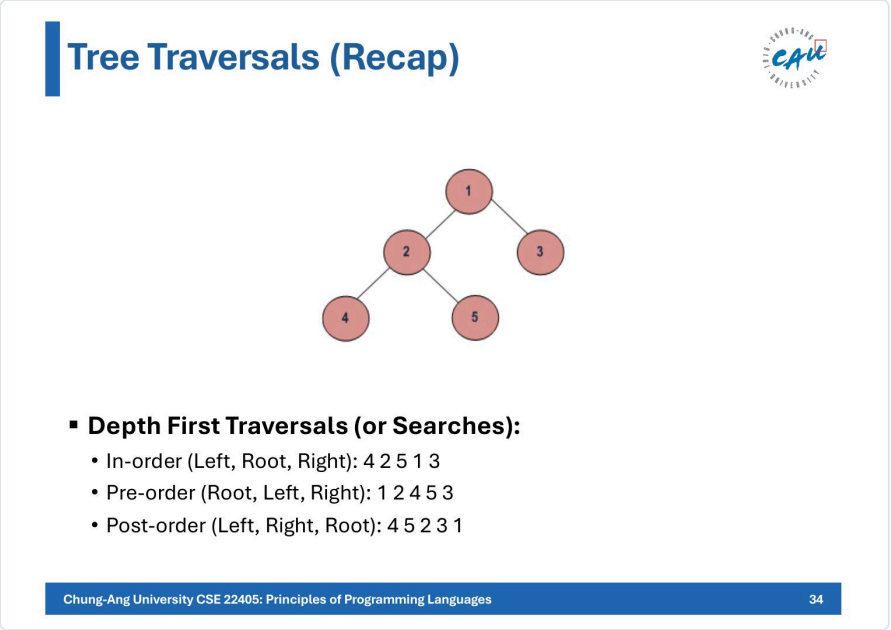
> 시험 포인트: 24 중간 7번. "하향식: 루트부터, 최좌단 유도 순서, 재귀하강/LL / 상향식: 잎부터, 최우단 유도의 역순, LR(shift-reduce)" 이 정도면 만점.

표기 관례 (Ch.4 전체에서 사용):

- 단말: 알파벳 앞쪽 소문자 `a, b, …`
- 비단말: 알파벳 앞쪽 대문자 `A, B, …`
- 단말 또는 비단말: 뒤쪽 대문자 `W, X, Y, Z`
- 단말 문자열: 뒤쪽 소문자 `w, x, y, z`
- 혼합 문자열: 그리스 소문자 `α, β, γ, δ`

예) 하향식: 문장 형태 `xAα`, 규칙 `A → bB | cBb | a`. 다음 토큰이 `c`면 `A → cBb` 선택.
예) 상향식: `S → aAc`, `A → aA | b`. 입력 `aabc`에서 `b`를 A로 환원 → `aaAc` → `aA`를 A로 → `aAc` → S. 이건 `S ⇒ aAc ⇒ aaAc ⇒ aabc` (최우단 유도)를 거꾸로 한 것.

> **수업 메모** 05주-1 · 05주-2
> 하향식은 루트에서 출발해 최좌단 유도 순서 = 전위 순회(root, left, right). 상향식은 입력에서 거꾸로 올라가므로 자연스럽게 최우단 유도의 역순이 된다.



L04 p.34 · 트리 순회 복습 (하향식 파서는 전위 순회 순서)

§ 파싱의 복잡도

- **모든** 모호하지 않은 문법에 작동하는 파서는 복잡하고 비효율적: **$O(n^3)$**.
- 그래서 컴파일러는 모호하지 않은 문법의 **부분 집합**에서만 동작하지만 **선형 시간 $O(n)$** 에 끝나는 파서를 쓴다.

> **수업 메모** `05주-1`
> 일반적인 파서는 주어진 토큰열에서 환원할 부분 문자열의 시작·끝 위치 후보가 너무 많아서 비효율적. 그래서 문법을 제한하고 선형 시간에 도는 파서를 쓴다.

재귀 하강 파서

교안 범위: L04 p.43–55

§ 재귀 하강 파서의 구조

- 부프로그램들의 모음. 재귀적이고, **하향식** 순서로 파스 트리를 만든다.
- 재귀는 프로그래밍 언어의 **중첩 구조**를 반영한 것.
- **문법의 비단말 하나당 부프로그램 하나.** 그 부프로그램은 그 비단말이 생성할 수 있는 문장을 파싱.
- **EBNF가 재귀 하강 파서의 기반으로 이상적**: EBNF는 비단말 수를 최소화하므로.

```
<expr>   → <term> {(+ | -) <term>}
<term>   → <factor> {(* | /) <factor>}
<factor> → id | int_constant | ( <expr> )
```

§ 코딩 규칙

어휘 분석기 `lex`가 다음 토큰 코드를 `nextToken`에 넣는다고 가정.

RHS가 하나일 때:

- RHS의 **단말**: 다음 입력 토큰과 비교 → 같으면 계속(lex 호출), 다르면 오류
- RHS의 **비단말**: 그 비단말의 파싱 부프로그램을 호출

```
void expr() {
    printf("Enter <expr>\n");
    term();
    while (nextToken == ADD_OP || nextToken == SUB_OP) {
        lex();
        term();
    }
    printf("Exit <expr>\n");
}
```

관례: 모든 파싱 루틴은 끝날 때 **다음 토큰을 nextToken에 남겨둔다.**

RHS가 여러 개인 비단말(예: `<factor>`): 다음 토큰(**lookahead**)을 각 RHS가 생성할 수 있는 **첫 토큰**과 비교해서 고른다. 맞는 게 없으면 구문 오류.

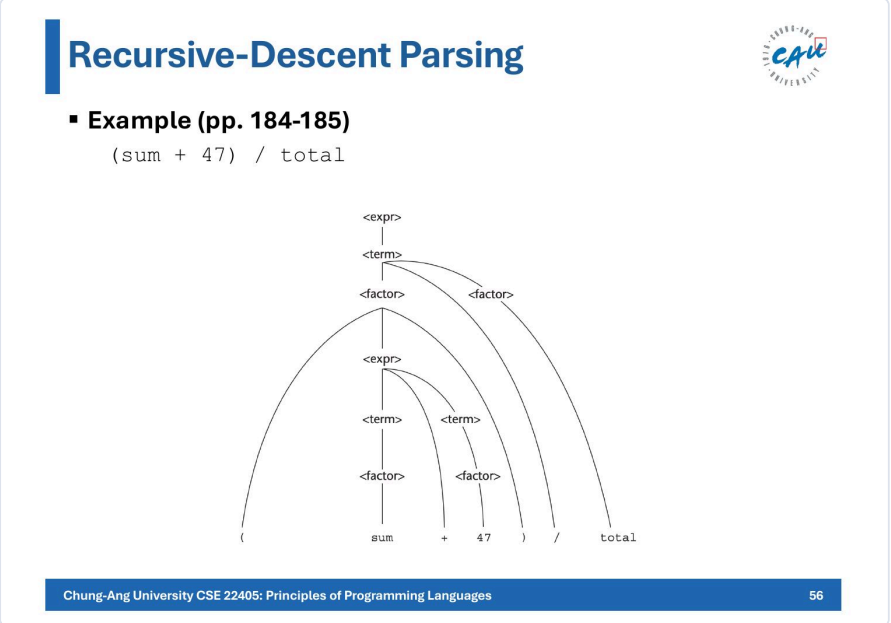
```
void factor() {
    printf("Enter <factor>\n");
    if (nextToken == IDENT || nextToken == INT_LIT)
        lex();
    else if (nextToken == LEFT_PAREN) {
        lex();
        expr();
        if (nextToken == RIGHT_PAREN) lex();
        else error();
    }
    else error();
    printf("Exit <factor>\n");
}
```

§ (sum + 47) / total 추적

```
Next token is: 25 Next lexeme is (
Enter <expr>
Enter <term>
Enter <factor>
Next token is: 11 Next lexeme is sum
Enter <expr>
Enter <term>
Enter <factor>
Next token is: 21 Next lexeme is +
Exit <factor>
Exit <term>
Next token is: 10 Next lexeme is 47
Enter <term>
Enter <factor>
Next token is: 26 Next lexeme is )
Exit <factor>
Exit <term>
Exit <expr>
Next token is: 24 Next lexeme is /
Exit <factor>
Next token is: 11 Next lexeme is total
Enter <factor>
Next token is: -1 Next lexeme is EOF
Exit <factor>
Exit <term>
Exit <expr>
```

읽는 법: `(`를 본 factor가 `lex()` 후 `expr()`을 재귀 호출 → 괄호 안을 다 파싱 → `)` 확인 후 `lex()` → `/`가 남아 있으니 term의 while이 한 번 더 돌아 factor 호출(total).

재귀 하강 파서는 **LL 파서**의 한 종류다 (Summary).



L04 p.55 · (sum + 47) / total 의 파스 트리

LL 문법: 좌순환 · 집합쌍 공통 · 좌추출

교안 범위: L04 p.56–62

§ LL 파싱

- **L**eft-to-right 스캔 + **L**eftmost derivation. lookahead 토큰으로 생성 규칙 선택.
- **LL 문법**: 왼→오로 읽으며 최좌단 유도를 구성하는 방식으로 파싱 가능한 CFG.
- LL(1) 파싱 테이블 예:

```
S → aAB     A → bA | ε     B → c

         a          b          c
S    S → aAB      error      error
A    error        A → bA     A → ε
B    error        error      B → c
```

하향식 파싱을 막는 문법의 두 가지 특성: ① **좌순환** ② **집합쌍 공통 테스트 실패**

§ 좌순환 문제와 제거

좌순환이 있으면(직접이든 간접이든) 하향식 파서의 기반이 될 수 없다.
이유: `A → A + B` 에서 A()가 아무 토큰도 소비하지 않고 바로 A()를 또 호출 → 무한 재귀.

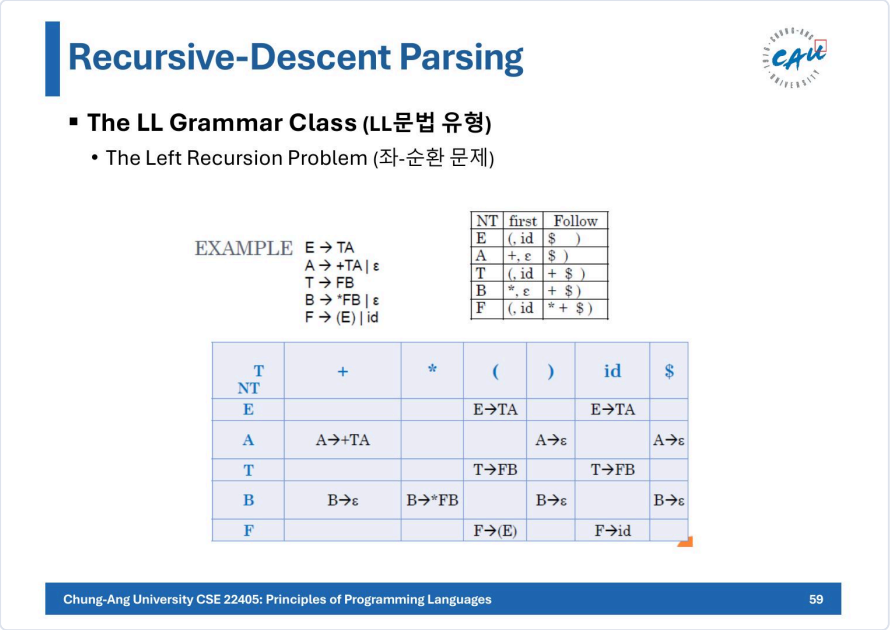
직접 좌순환 제거:

```
A → Aα1 | … | Aαm | β1 | β2 | … | βn     (β들은 A로 시작하지 않음)
⇓
A  → β1A' | β2A' | … | βnA'
A' → α1A' | α2A' | … | αmA' | ε
```

예)

```
E → E + T | T            ⇒   E  → T E'
                             E' → + T E' | ε
T → T * F | F            ⇒   T  → F T'
                             T' → * F T' | ε
F → (E) | id             ⇒   그대로
```

요령: **좌순환 대안의 꼬리(α)는 A'로, 좌순환이 아닌 대안(β)은 뒤에 A'를 붙여서 A로.**



L04 p.58 · 좌순환 제거 예 (E → TE', E' → +TE' | ϵ ...)와 LL 테이블

§ 집합쌍 공통 테스트 (Pairwise Disjointness Test)

하향식 파싱을 막는 다른 특성 = 다음 토큰 하나로 올바른 RHS를 고를 수 없는 것.

FIRST(α) = { a | α ⇒* aβ } : α에서 유도되는 문자열의 첫 단말들의 집합. (α ⇒* ϵ 이면 ϵ도 포함)

테스트: 둘 이상의 RHS를 가진 각 비단말 A에 대해, 모든 규칙 쌍 `A → αi`, `A → αj` 에 대해
FIRST(αi) ∩ FIRST(αj) = ∅ 이어야 통과.

예)

```
A → a | bB | cAb
  FIRST(a) = {a}, FIRST(bB) = {b}, FIRST(cAb) = {c}  → 모두 서로소 → 통과
A → a | aB
  FIRST(a) = {a}, FIRST(aB) = {a}  → 공통 a → 실패
```

RHS가 **비단말로 시작**하면 그 비단말의 FIRST를 구해서 넣어야 한다:

```
A → aB | Bb ,  B → aC | d
  FIRST(aB) = {a},  FIRST(Bb) = FIRST(B) = {a, d}  → 공통 a → 실패
```

시험 포인트: 24 중간 13번(5점). 답안 형식 = **각 RHS의 FIRST 집합을 모두 쓰고 → 교집합 → 통과/실패 판정**. 결론만 쓰면 감점.

수업 메모 05주-2
FIRST가 겹친다 = 같은 첫 토큰으로 시작하는 유도가 둘 이상 있다는 뜻 → 다음 토큰 하나만 보고 규칙을 고를 수 없다. 해결책 중 하나가 좌추출.

§ 좌추출 (Left factoring)

집합쌍 공통 테스트 실패를 해결할 수 있는 경우가 있다: 공통 접두부를 뽑아낸다.

```
<variable> → identifier | identifier [<expression>]
⇓
<variable> → identifier <new>
<new>      → ε | [<expression>]
또는 (EBNF)
<variable> → identifier [[<expression>]]     (바깥 [ ]는 EBNF 메타기호)
```

일반형: `A → αβ1 | αβ2` ⇒ `A → αA'`, `A' → β1 | β2`

U6 · L04 p.63–79

Ch.4 상향식 파싱 (핸들 · LR)

24 중간 14번(핸들·어구·단순어구 5점)과 15번(LR 파싱 추적 10점). LR 추적은 손이 기억해야 하니 퀴즈에서 단계별로 계속 시킬 거야.

핸들 · 어구 · 단순 어구

교안 범위: L04 p.63–67

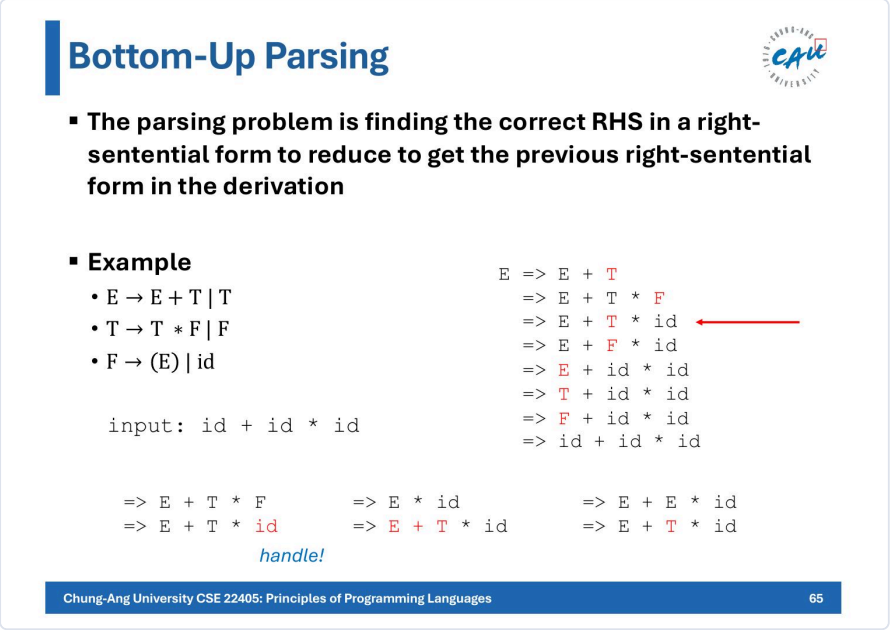
§ 상향식 파싱의 문제

우문장 형태(right sentential form, 최우단 유도에서 나오는 문장 형태)에서, 환원하면 **바로 이전 우문장 형태**가 되는 RHS 부분 문자열을 찾는 것.

```
E → E + T | T       T → T * F | F       F → (E) | id
입력: id + id * id

최우단 유도:  E ⇒ E + T ⇒ E + T * F ⇒ E + T * id ⇒ E + F * id
            ⇒ E + id * id ⇒ T + id * id ⇒ F + id * id ⇒ id + id * id
상향식 파서는 이걸 아래에서 위로 거슬러 올라간다.
```

예) `E + T * id` 에서 환원할 수 있는 RHS 후보: `E + T`(→E), `T`(→E), `id`(→F).
그런데 이전 우문장 형태 `E + T * F`가 되려면 `id` 를 F로 환원해야 함. 이 `id`가 **핸들**.



L04 p.64 · id + id * id 의 최우단 유도와 각 단계에서 환원할 핸들

§ 정의 3개

우문장 형태 γ에 대해:

- **핸들** (handle): βα가 γ = αβw 의 핸들 ⇔ `S ⇒*rm αAw ⇒rm αβw`
  → 최우단 유도에서 **바로 직전 단계에 A가 β로 전개된 바로 그 부분**.
- **어구** (phrase): βα가 γ의 어구 ⇔ `S ⇒* γ = α1Aα2 ⇒+ α1βα2`
  → 파스 트리에서 **어떤 내부 노드 하나를 루트로 하는 부분 트리의 잎들** (1단계 이상).
- **단순 어구** (simple phrase): `S ⇒* α1Aα2 ⇒ α1βα2` (**한 단계**)
  → 내부 노드 A의 자식들이 **전부 잎**인 경우, 그 자식들.

핵심 관계:

- **핸들 = 가장 왼쪽의 단순 어구** (the leftmost simple phrase)

- 파스 트리가 있으면 핸들 찾기는 쉽다.
- 상향식 파싱 = **핸들 가지치기** (handle pruning)

---

§ 파스 트리로 푸는 법

`E + T * id` 의 파스 트리:

```
        E
      / | \
     E  +  T
         / | \
        T  *  F
              |
             id
```

1. 내부 노드마다 그 아래 **잎들을 왼→오로 읽은 것** = 어구
   - E(루트): `E + T * id` - T: `T * id` - F: `id`
2. 자식이 **전부 잎**인 내부 노드 = 단순 어구 → F의 자식 `id`만 해당 (T의 자식 중 F는 내부 노드라 X)
3. 단순 어구 중 **가장 왼쪽** = 핸들 → `id`

답: 어구 = {E + T * id, T * id, id}, 단순 어구 = {id}, 핸들 = id

실수 주의: 루트(시작 기호) 전체 문자열도 **어구**에 포함. 잎이 비단말이어도(예: 위 트리의 E, T) 그건 잎이다 — "잎"은 트리에서 자식이 없는 노드. 시험 답안엔 **파스 트리를 반드시 그려**.

연습 예) `S → aAb | bBA`, `A → ab | aAB`, `B → aB | b`, 우문장 형태 `aaAbb`:

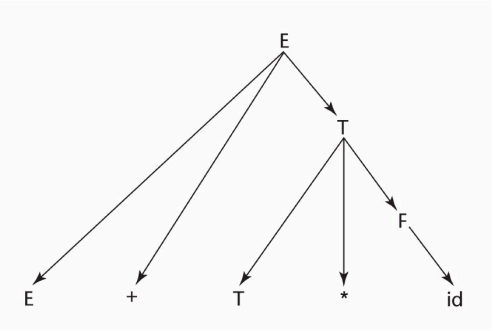
```
        S
      / | \
     a  A  b
      / | \
     a  A  B
           |
           b
```

어구: `aaAbb`(S), `aAb`(A), `b`(B의 자식, 4번째 글자) / 단순 어구: `b` / 핸들: `b`

> **수업 메모** 06주-2
> 단순 어구를 답할 때 같은 기호가 여러 번 나오면(예: id가 두 개) 어느 위치의 것인지 반드시 명시하라고 함. 핸들은 단순 어구 중 가장 왼쪽 것 = 최우단 유도에서 마지막으로 전개된 부분.

Bottom-Up Parsing

- **Example**
  - E + T * id



L04 p.66 · 우문장 형태 E + T * id 의 파스 트리

Shift-Reduce · LR 파서 구조

교안 범위: L04 p.68–76

§ Shift와 Reduce

상향식 파서 = **shift-reduce 알고리즘**:

- **Shift**: 다음 입력 토큰을 파스 스택의 top으로 옮김
- **Reduce**: 스택 top에 있는 **핸들**을 대응하는 LHS로 교체

§ LR 파서의 장점

1. 프로그래밍 언어를 기술하는 **거의 모든 문법**에 대해 동작
2. 다른 상향식 알고리즘보다 **더 넓은 문법 클래스**에서 동작하면서도 **다른 상향식 파서만큼 효율적**
3. 구문 오류를 **가능한 한 빨리** 탐지
4. **LR 문법 클래스는 LL 파서로 파싱 가능한 클래스의 상위 집합**

단점(교안): LR 파싱 테이블은 손으로 만들기 어려워 **도구**로 생성 (yacc, bison).

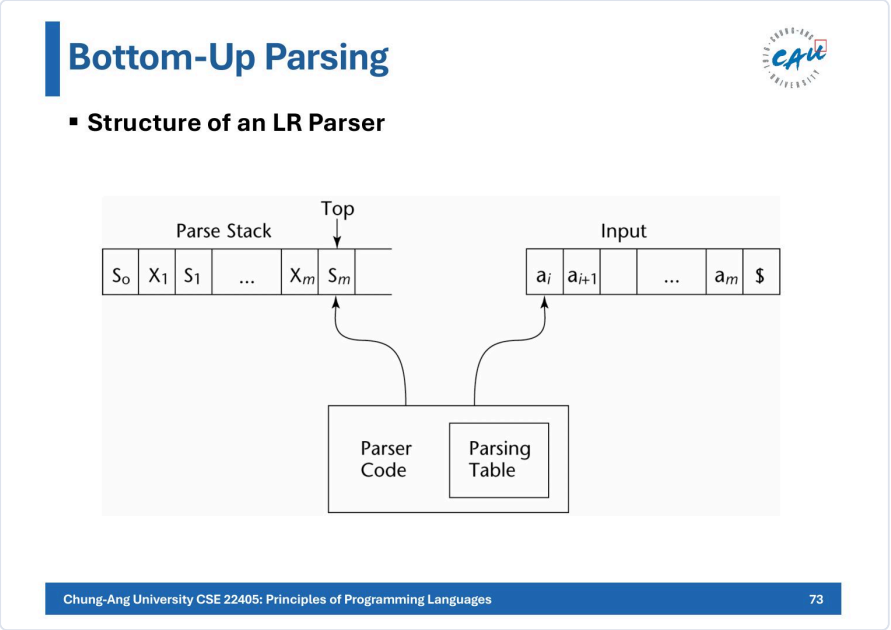
Knuth의 통찰: 상향식 파서는 현재까지의 **파싱 이력 전체**를 써서 결정할 수 있다. 가능한 파싱 상황의 수는 유한하고 비교적 작아서, 이력을 **파서 상태**로 스택에 저장할 수 있다.

> **수업 메모** 05주-2
> LR은 지금까지 읽은 이력을 스택(상태)에 누적해서 판단하기 때문에 LL보다 넓은 범위의 CFG를 다룰 수 있다 (좌순환이나 집합 쌍 테스트 실패 문법도 처리 가능).

§ LR 파서의 구성

- 파스 스택: 상태 기호와 문법 기호를 번갈아 저장
- **LR 구성** (configuration): `(S0 X1 S1 X2 S2 … Xm Sm, ai ai+1 … an $)`
  왼쪽 = 스택, 오른쪽 = 남은 입력
- 초기 구성: `(S0, a1 … an $)`
- 파싱 테이블 = **ACTION 테이블 + GOTO 테이블**

테이블	행	열	내용
ACTION	상태	**단말**	주어진 상태와 다음 토큰에서 할 행동 (Shift/Reduce/Accept/Error)
GOTO	상태	**비단말**	**reduce 후** 스택에 넣을 상태



L04 p.72 · LR 파서의 구조 (파스 스택 + 입력 + 파서 코드 + 파싱 테이블)

§ 파서 행동 4가지

- **Shift (Sn)**: 다음 입력 기호와, Shift에 적힌 상태 n을 스택에 push.
- **Reduce (Rn)**: 규칙 n의 핸들(RHS)을 스택에서 제거 — **RHS 기호 수 × 2개**(기호+상태)를 pop. 그다음 LHS를 push하고, **새 LHS 바로 밑에 있는 상태**와 **LHS**로 GOTO 테이블을 찾아 그 상태를 push.
- **Accept**: 파싱 완료, 오류 없음.
- **Error**: 오류 처리 루틴 호출.

> 시험에서 가장 많이 틀리는 곳: Reduce 후 GOTO를 찾을 때 쓰는 상태는 **pop하고 남은 스택 top의 상태**다. reduce 전 상태가 아님.

LR 파싱 추적 연습

교안 범위: L04 p.77–78

§ 시험에 주어지는 문법과 파싱 테이블

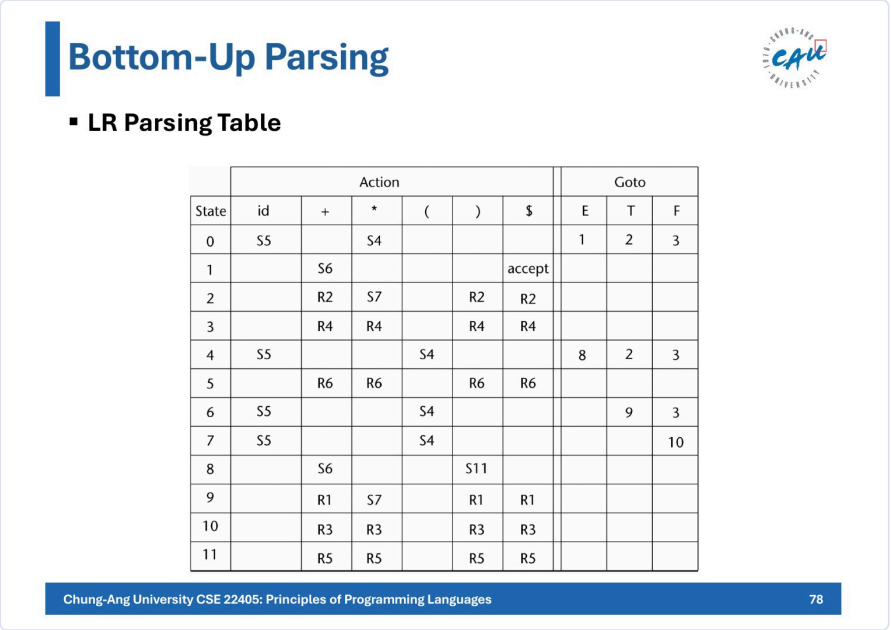
```
1. E → E + T      4. T → F
2. E → T          5. F → (E)
3. T → T * F      6. F → id
```

(교안 p.78 규칙 5가 `F → E`로 오타가 나 있는데 실제로는 `F → (E)`. 24 중간 시험지는 `(E)`로 맞게 나왔어.)

상태	id	+	*	(	)	$	E	T	F
0	S5			S4			1	2	3
1		S6				accept			
2		R2	S7		R2	R2			
3		R4	R4		R4	R4			
4	S5			S4			8	2	3
5		R6	R6		R6	R6			
6	S5			S4				9	3
7	S5			S4					10
8		S6			S11				
9		R1	S7		R1	R1			
10		R3	R3		R3	R3			
11		R5	R5		R5	R5			

수업 메모 06주-2

ACTION/GOTO 테이블은 시험에서 주어진다. 이 수업은 컴파일러 수업이 아니라 테이블을 만드는 법은 다루지 않고 (yacc 같은 도구가 만들어 줌), 테이블을 읽어서 파싱 과정을 따라가는 것만 하면 된다.



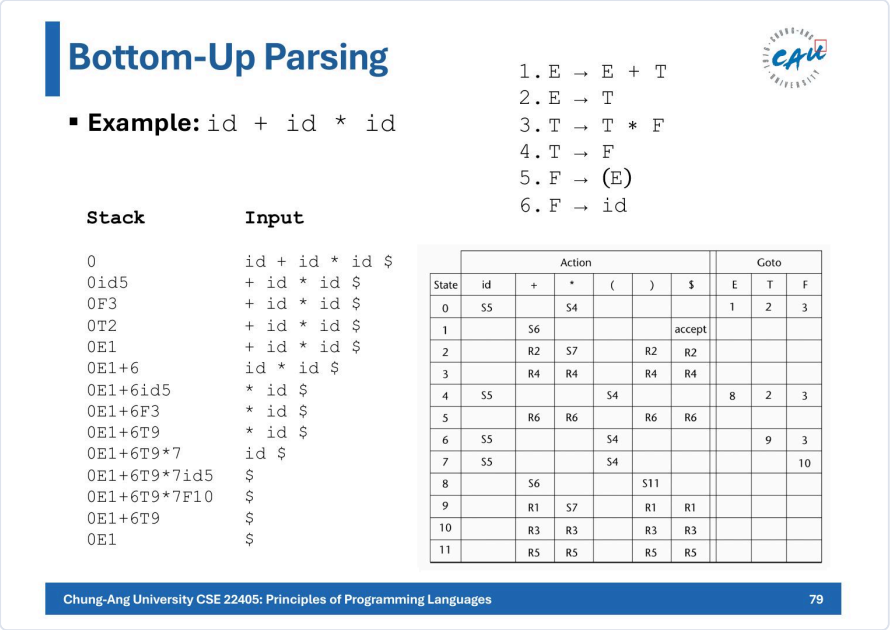
	Action						Goto		
State	id	+	*	(	)	$	E	T	F
0	S5		S4				1	2	3
1		S6				accept			
2		R2	S7		R2	R2			
3		R4	R4		R4	R4			
4	S5			S4			8	2	3
5		R6	R6		R6	R6			
6	S5			S4				9	3
7	S5			S4					10
8		S6			S11				
9		R1	S7		R1	R1			
10		R3	R3		R3	R3			
11		R5	R5		R5	R5			

L04 p.77 · LR 파싱 테이블 (ACTION / GOTO)

§ 교안 예제: id + id * id

스택	입력	행동
0	id + id * id $	Shift 5
0id5	+ id * id $	Reduce 6 (F→id), GOTO[0,F]=3
0F3	+ id * id $	Reduce 4 (T→F), GOTO[0,T]=2
0T2	+ id * id $	Reduce 2 (E→T), GOTO[0,E]=1
0E1	+ id * id $	Shift 6
0E1+6	id * id $	Shift 5
0E1+6id5	* id $	Reduce 6, GOTO[6,F]=3
0E1+6F3	* id $	Reduce 4, GOTO[6,T]=9
0E1+6T9	* id $	Shift 7
0E1+6T9*7	id $	Shift 5
0E1+6T9*7id5	$	Reduce 6, GOTO[7,F]=10
0E1+6T9*7F10	$	Reduce 3 (T→T*F), GOTO[6,T]=9
0E1+6T9	$	Reduce 1 (E→E+T), GOTO[0,E]=1
0E1	$	Accept

수업 메모 06주-2
수업에서 id + id * id를 칠판에 한 줄씩 따라감: 스택 맨 위 상태와 입력 첫 토큰으로 ACTION을 보고, Reduce 후엔 남은 스택 top 상태와 LHS로 GOTO를 본다. 상태 1에서 $를 만나면 accept, 테이블에 없는 칸은 전부 error.



Bottom-Up Parsing

- **Example:** id + id * id

1. E → E + T
2. E → T
3. T → T * F
4. T → F
5. F → (E)
6. F → id

Stack	Input
0	id + id * id $
0id5	+ id * id $
0F3	+ id * id $
0T2	+ id * id $
0E1	+ id * id $
0E1+6	id * id $
0E1+6id5	* id $
0E1+6F3	* id $
0E1+6T9	* id $
0E1+6T9*7	id $
0E1+6T9*7id5	$
0E1+6T9*7F10	$
0E1+6T9	$
0E1	$

	Action						Goto		
State	id	+	*	(	)	$	E	T	F
0	S5		S4				1	2	3
1		S6				accept			
2		R2	S7		R2	R2			
3		R4	R4		R4	R4			
4	S5			S4			8	2	3
5		R6	R6		R6	R6			
6	S5			S4				9	3
7	S5			S4					10
8		S6			S11				
9		R1	S7		R1	R1			
10		R3	R3		R3	R3			
11		R5	R5		R5	R5			

Chung-Ang University CSE 22405: Principles of Programming Languages 79

L04 p.78 · id + id * id 의 LR 파싱 과정 (규칙 5는 F → (E) 가 맞음)

§ 한 줄씩 푸는 루틴

매 줄마다 이것만 반복:

1. 스택 **맨 오른쪽 상태 번호**와 입력 **맨 앞 토큰**으로 ACTION 칸을 본다.
2. `Sn` → 토큰과 n을 스택 끝에 붙이고, 입력에서 토큰을 지운다.
3. `Rn` → 규칙 n의 RHS 길이 k만큼 (기호, 상태) 쌍을 스택 끝에서 지운다 → 남은 스택 끝 상태 s와 LHS로 GOTO[s, LHS] → `LHS` + 그 상태를 붙인다. **입력은 그대로.**
4. `accept` → 끝.

체크 포인트:

- `R3 (T→T*F)` 은 **3개 기호**(T, *, F) = 6칸 pop. `R1 (E→E+T)` 도 3개. `R5 (F→(E))` 도 3개(`(` , E, `)`).
- `R2, R4, R6` 는 1개 기호 = 2칸 pop.
- 상태 2에서 `*` 를 보면 **S7**(곱셈이 우선), `+` 를 보면 **R2**. 상태 9도 같다. → 이게 우선순위가 표에 녹아 있는 방식.

U7 · 교안 미배포 (10/13 수업 예정) · Sebesta Ch.5 기준

Ch.5 이름 · 바인딩 · 영역

다음 주 화요일에 나갈 범위인데 24 중간 5·16·17·18번(33점)이 전부 여기였어. 교재(Sebesta) 기준으로 정리해 뒀고, 수업 교안이 나오면 맞춰서 고칠게.

이름 · 변수의 6가지 속성

교안 범위: Sebesta 5.2–5.3

§ 이름 (Names)

설계 이슈:

- **대소문자 구분**(case sensitivity)? C 계열은 구분 → `rose`, `ROSE`, `Rose`가 다른 이름 → 판독성 저하.
- **특수어**(special words)가 **예약어**(reserved word)인가 **키워드**(keyword)인가?
  - 키워드: 특정 문맥에서만 특별한 의미 (Fortran의 `Integer`는 변수 이름도 될 수 있었음)
  - 예약어: 이름으로 **사용 불가**. 예약어가 너무 많으면 이름 짓기 곤란 (COBOL 300개)
- 이름 길이 제한, 허용 문자 (PHP 변수는 `$`로 시작, Perl은 `$ @ %`로 타입 표시, Ruby `@`, `@@`)

§ 변수의 6가지 속성

변수 = 메모리 셀(들)의 추상화. 6개 속성의 **6-tuple**로 특징지어짐:

속성	설명
이름 (name)	변수를 가리키는 식별자. 모든 변수가 이름을 갖진 않음(힙의 익명 변수)
주소 (address)	변수와 연관된 메모리 주소 = **l-value**. 실행 중 다른 시점에 다른 주소일 수 있음(재귀 호출의 지역 변수)
값 (value)	변수 메모리 셀의 내용 = **r-value**
타입 (type)	변수 값의 범위와 그 타입에 정의된 연산의 집합을 결정
생존 기간 (lifetime)	변수가 특정 메모리 셀에 **바인딩되어 있는 기간**
영역 (scope)	변수가 **가시적인**(참조 가능한) 문장들의 범위

> 시험 포인트: 24 중간 5번 = 6가지 속성 적기(3점). 외우는 법: **이주값타생영** (이름·주소·값·타입·생존기간·영역).

별칭 (aliases): 둘 이상의 변수 이름이 **같은 메모리 주소**에 접근할 때. 포인터, 참조 변수, C/C++ union으로 생김. 판독성·신뢰성에 해롭다(한 이름으로 값을 바꾸면 다른 이름의 값도 바뀜).

바인딩 · 바인딩 시간 · 타입 바인딩

교안 범위: Sebesta 5.4.1–5.4.2

§ 바인딩과 바인딩 시간

바인딩 (binding): 속성과 개체 사이의 연관 (예: 변수와 그 타입/값, 연산과 기호).
바인딩 시간: 바인딩이 일어나는 시점.

바인딩 시간	예
언어 설계 시간	`*` 기호를 곱셈 연산에 바인딩
언어 구현 시간	C의 `int` 같은 부동소수점/정수 타입을 표현 방식에 바인딩
컴파일 시간	C/Java 변수를 특정 데이터 타입에 바인딩
적재(load) 시간	C/C++ `static` 변수를 메모리 셀에 바인딩
링크 시간	라이브러리 부프로그램 호출을 그 코드에 바인딩
실행 시간	비정적 지역 변수를 메모리 셀에 바인딩

`count = count + 5;` 에서: count의 타입 → 컴파일 시간 / `+`의 가능한 의미 → 언어 설계 시간 / `+`의 실제 의미 → 컴파일 시간(피연산자 타입이 정해지면) / 5의 내부 표현 → 구현 시간 / count의 값 → 실행 시간.

> **수업 메모** 06주-2
> 10/8 예고: 다음 주(7주차)에 Ch.5 이름·바인딩·영역. 바인딩 = 변수를 선언할 때 식별자와 타입 등 속성을 연결하는 것. 형식 매개변수(함수 정의의 매개변수)도 영역 관점에서 다룰 예정.

§ 정적 바인딩 vs 동적 바인딩

- **정적 바인딩**: **실행 시간 이전**에 처음 일어나고 실행 내내 **변하지 않음**.
- **동적 바인딩**: **실행 중**에 처음 일어나거나 실행 중에 **변할 수 있음**.

§ 타입 바인딩

정적 타입 바인딩 방법:

- **명시적 선언** (explicit declaration): 변수 이름과 타입을 나열하는 문장. `int count;`
- **묵시적 선언** (implicit declaration): 선언문 없이 **기본 관례**로 타입을 정함.
  - 이름 관례: Fortran은 I, J, K, L, M, N으로 시작하면 정수, 나머지는 실수. Perl은 `$`(스칼라) `@`(배열) `%`(해시).
  - **타입 추론**(type inference): 문맥으로 추론. C#의 `var sum = 0;`, ML, Haskell.
  - 단점: 오타가 새 변수가 되어 오류가 안 잡힐 수 있음 → **신뢰성 저하** (이름 관례 방식은 덜 심함).

동적 타입 바인딩 (JavaScript, Python, Ruby, PHP):

```
list = [2, 4.33, 6, 8];    // list는 배열
list = 17.3;                // 이제 list는 스칼라
```

- 배정문으로 타입이 정해짐 (배정되는 값의 타입)
- 장점: **유연성** (generic 프로그램 작성 쉬움)

- 단점: **비용이 큼** (실행 시간 타입 검사, 보통 순수 해석으로 구현), **타입 오류 탐지가 어려움** (잘못된 타입의 값이 배정돼도 오류가 아님 → 신뢰성↓)

기억장소 바인딩과 생존 기간

교안 범위: Sebesta 5.4.3

§ 할당 · 해제 · 생존 기간

- **할당** (allocation): 가용 메모리 풀에서 셀을 가져와 변수에 바인딩
- **해제** (deallocation): 바인딩을 끊고 셀을 풀로 돌려줌
- **생존 기간** (lifetime): 변수가 특정 메모리 셀에 바인딩되어 있는 시간

§ 생존 기간에 따른 변수 4종류

종류	바인딩 시점	예	장점	단점
정적 (static)	실행 **전**에 메모리에 바인딩, 종료까지 유지	C/C++ `static` 변수, 전역 변수	효율적(직접 주소 지정), **이력 민감**(history-sensitive) 부프로그램 지원	유연성↓, **재귀 불가**, 변수 간 저장 공간 공유 불가
스택-동적 (stack-dynamic)	선언문이 **세련화(elaboration)** 될 때 (실행 시간) 바인딩. 타입은 정적	C/Java 메소드의 지역 변수	**재귀** 지원, 부프로그램 간 메모리 공유	할당/해제 오버헤드, 간접 주소 지정, 이력 민감 X
명시적 힙-동적 (explicit heap-dynamic)	프로그래머가 **명시적 명령** (new/delete, malloc/free)으로 할당·해제. **포인터/참조**로만 접근	C++ `new int`, Java 객체	동적 자료 구조(연결 리스트, 트리)	포인터 사용이 어렵고 **신뢰성↓**, 힙 관리 비용
묵시적 힙-동적 (implicit heap-dynamic)	**배정될 때** 힙에 할당 (모든 속성이 매번 새로 바인딩)	JavaScript 배열·문자열, Perl	유연성 최고	비효율(모든 속성이 동적), 오류 탐지 상실

세련화 (elaboration): 선언문이 실행 시간에 처리되어 기억장소를 할당하고 바인딩하는 것. 보통 함수 진입 시 일어남.

함정: Java의 객체는 **명시적** 힙-동적 (new로 만듦, 해제는 GC가 묵시적으로). C++의 `static` 지역 변수는 영역은 지역이지만 생존 기간은 프로그램 전체 → 다음 레슨의 "영역 vs 생존 기간"과 연결.

정적 영역 · 블록 · 전역 영역

교안 범위: Sebesta 5.5.1–5.5.4

§ 영역 기본 용어

- **영역** (scope): 변수가 **가시적인**(visible) 문장들의 범위. 문장에서 참조할 수 있으면 가시적.
- **지역 변수** (local): 그 프로그램 단위/블록 안에서 선언된 변수
- **비지역 변수** (nonlocal): 가시적이지만 그 단위 안에서 선언되지 않은 변수
- **전역 변수** (global): 비지역 변수의 특수한 경우

§ 정적 영역 (Static scoping)

- 영역이 **프로그램 텍스트(구조)** 로 결정 → **실행 전**(컴파일 시간)에 결정 가능. ALGOL 60에서 도입.
- 참조된 이름의 선언을 찾는 순서: **지역 → 정적 부모(static parent, 자신을 감싸는 단위) → 그 부모 → … → 전역**
- **정적 조상** (static ancestors): 정적 부모들의 연쇄
- 가까운 영역의 같은 이름이 바깥 선언을 **가린다** (hidden). C++은 `::x`로 가려진 전역 접근, Ada는 `unit.x`.

예) (JavaScript, 함수 중첩)

```
function big() {
    function sub1() {
        var x = 7;
        sub2();
    }
    function sub2() {
        var y = x;      // 이 x는?
    }
    var x = 3;
    sub1();
}
```

정적 영역: sub2의 정적 부모는 **big** → `x`는 **big의 x (= 3)**. (sub1이 sub2를 호출했어도 상관없음)

§ 블록과 선언 순서

블록 (block): 코드 중간에 새로운 정적 영역을 만드는 방법 (ALGOL 60). C 계열의 `{ }`.

```
void sub() {
    int count;
    while (...) {
        int count;      // 바깥 count를 가림
        count++;        // 안쪽 count
    }
    ...                 // 바깥 count
}
```

- C/C++: 중첩 블록에서 같은 이름 재선언 **허용**(가림). **Java, C#은 금지**(오류 가능성이 높아서).
- 블록 안 변수는 보통 **스택-동적** (블록 진입 시 할당, 나올 때 해제).

선언 순서: C99, C++, Java는 블록 어디서나 선언 가능 → 영역은 **선언부터 블록 끝까지**. C#은 블록 **어디서 선언하든 블록 전체**가 영역(단 사용 전에 선언돼야 함).

`for (int i = 0; …)` 의 i: C++/Java/C#에서 영역은 **for문 안**으로 한정.

블록 가시성 문제 푸는 법 (24 중간 17번 유형): 각 지점에서 **안쪽 블록 → 바깥 블록** 순서로 선언을 찾되, 같은 이름은 **가장 안쪽 것만** 보인다. 이미 닫힌 블록의 변수는 안 보인다.

---

§ 전역 영역

- C/C++/PHP/Python/JavaScript: 함수 바깥에 변수 정의를 둘 수 있음 → 전역.
- C: **선언**(declaration, 타입·속성만, 기억장소 X: `extern int sum;`) vs **정의**(definition, 기억장소 할당).
- PHP: 함수 안에서 전역 변수를 쓰려면 `global $x;` 또는 `$GLOBALS['x']`.
- Python: 전역 변수를 함수 안에서 **배정**하려면 `global x` 선언 필요 (안 하면 지역 변수로 간주 → 배정 전 참조 시 오류).

---

§ 정적 영역 평가

대부분의 경우 잘 동작하지만:

- 변수·부프로그램에 **필요 이상으로 많은 접근**이 허용됨
- 프로그램이 진화하면서 초기 구조가 무너짐 → 전역 변수가 늘어나는 경향
- 대안: **캡슐화 구조** (Ch.11)

동적 영역 · 영역 vs 생존 기간 · 참조 환경

교안 범위: Sebesta 5.5.5–5.8

§ 동적 영역 (Dynamic scoping)

- 영역이 프로그램 텍스트가 아니라 **부프로그램의 호출 순서**(calling sequence)로 결정 → **실행 시간**에만 결정 가능. (APL, SNOBOL4, 초기 LISP, Perl의 `local`)
- 비지역 참조는 **가장 최근에 호출되어 아직 활성화된 부프로그램**부터 **호출 체인을 거꾸로** 따라가며 찾는다.

앞 레슨의 big/sub1/sub2 예:

- 호출 순서 big → sub1 → sub2 : sub2에 x 없음 → 호출자 **sub1의 x (= 7)**
- 호출 순서 big → sub2 : sub2에 x 없음 → 호출자 **big의 x (= 3)**

동적 영역 가시성 푸는 법 (24 중간 18번 유형):

1. 호출 체인의 **마지막 함수부터 main 쪽으로** 거슬러 올라가며
2. 각 함수의 지역 변수를 적되, **이미 적은 이름은 건너뛴다** (가려짐)

예) `main: var a, b; sub1: var b, c; sub2: var c, d; sub3: var a, d, e`
main → sub1 → sub2 → sub3 일 때 마지막(sub3) 실행 중 가시 변수:

```
sub3: a, d, e           (sub3 것)
sub2: c                 (d는 sub3이 가림)
sub1: b                 (c는 이미 있음)
main: (a, b 다 가려짐) → 없음
답: a(sub3), d(sub3), e(sub3), c(sub2), b(sub1)
```

동적 영역 평가:

- 장점: **편리함** (호출된 부프로그램에 매개변수 없이 변수 전달)
- 단점:
  - 부프로그램 실행 중 그 지역 변수가 **실행 중인 다른 모든 부프로그램에 보인다** → 신뢰성↓
  - 비지역 변수에 대한 **정적 타입 검사 불가능**
  - **판독성 나쁨**: 호출 순서를 알아야 참조 대상을 알 수 있음
  - 비지역 변수 접근이 정적 영역보다 **느림**

---

§ 영역 vs 생존 기간

둘은 관련 있어 보이지만 **다른 개념**:

- 영역 = **공간적**(텍스트) 개념, 생존 기간 = **시간적** 개념.

```
void someSub() {
    static int count = 0;    // 영역: someSub 안 (지역)
    ...                      // 생존 기간: 프로그램 실행 전체
}
```

```
void printheader() { ... }
void compute() {
    int sum;
    printheader();          // printheader 실행 중 sum은 영역 밖(안 보임)
}                           // 하지만 sum은 살아 있음(생존 기간 안)
```

§ 참조 환경

참조 환경 (referencing environment): 어떤 문장에서 **가시적인 모든 이름의 집합**.

- **정적 영역 언어**: 지역 변수 + **모든 정적 조상**의 가시 변수
- **동적 영역 언어**: 지역 변수 + **현재 활성화된 모든 부프로그램**의 가시 변수
- **활성화**(active) 부프로그램: 실행을 시작했지만 아직 종료하지 않은 부프로그램

§ 이름 상수

이름 상수 (named constant): 값에 **한 번만** 바인딩되는 변수.

- 장점: **판독성**(3.14159 대신 pi), **수정 용이성**(파라미터화: 상수 하나만 바꾸면 전체 반영)
- 값 바인딩: 정적(컴파일 시간, C++ `const`에 상수식, C# `const`) 또는 동적(실행 시간에 변수 포함식으로 초기화 가능, Java `final`, C# `readonly`)